



SE507 - SOFTWARE TESTING AND EVALUATION

A Comprehensive Testing Strategy for a Library Management System

Team Members:

Tishko Salah (Test Manager)
Sara Zuhair (Code specifier)
Yassin Hussein (Code reviewer)
Ravyar Barzan (Document inspector)

Module Leader: **Dr. Hossein Hassani**

June 16, 2025

University of Kurdistan Hewlêr
Department of Computer Science and Engineering
Erbil, Kurdistan Region, Iraq

Contents

1	Introduction	6
1.1	Problem Domain and Project Rationale	6
1.2	Purpose and Contribution of This Report	7
1.3	Scope of the Testing Mandate	7
1.4	System Architecture Overview	8
2	Background	9
2.1	System Under Test (SUT) Overview	9
2.2	System Architecture	9
2.3	User Roles and Functionalities	10
2.3.1	Librarian (Admin)	10
2.3.2	Member	10
2.4	Data Model	11
3	Test Management	12
3.1	Test Strategy	12
3.1.1	Overall Approach	12
3.1.2	Test Levels	12
3.2	Test Scope	13
3.2.1	In Scope	13
3.2.2	Out of Scope	13
3.3	Team Roles and Responsibilities	14
3.4	Test Environment and Tools	14
3.5	Test Schedule	15
3.6	Deliverables	15
3.7	Risk Management	15
3.8	Entry and Exit Criteria	16
3.8.1	Entry Criteria	16
3.8.2	Exit Criteria	16
4	Testing and Evaluation Process	18
4.1	Testing Strategy Overview	18
4.2	Testing Levels	19
4.2.1	Unit Testing	20
4.2.1.1	Authentication Controller (authController)	20
4.2.1.2	Author Controller (authorController)	21
4.2.1.3	Book Controller (bookController)	22
4.2.1.4	Borrowal Controller (borrowalController)	23
4.2.1.5	Genre Controller (genreController)	24
4.2.1.6	Review Controller (reviewController)	24
4.2.2	Integration Testing	25

4.2.3	System Testing	29
4.3	Dynamic Testing Types	29
4.3.1	Functional Testing	29
4.3.1.1	Authentication and Authorization	30
4.3.1.2	Entity Management Testing	34
4.3.1.3	Use Case Testing	41
4.3.1.4	State Transition Testing	44
4.3.1.5	Specific Feature Testing	47
4.3.2	API Testing	52
4.3.3	White-Box Testing	56
4.3.3.1	Test Strategy and Coverage Criteria	56
4.3.3.1.1	Coverage Types Implemented	56
4.3.3.1.2	Statement Coverage	56
4.3.3.1.3	Branch Coverage	56
4.3.3.1.4	Condition Coverage	57
4.3.3.1.5	Path Coverage	57
4.3.3.2	Complexity Analysis	57
4.3.3.2.1	Cyclomatic Complexity Assessment	57
4.3.3.3	Server-Side White-Box Testing	57
4.3.3.3.1	Model Testing	57
4.3.3.3.1.1	User Model (TC_WB_USER_001 - TC_WB_USER_017)	57
4.3.3.3.1.2	Book Model (TC_WB_BOOK_MODEL_001 - TC_WB_BOOK_MODEL_026)	58
4.3.3.3.2	Controller Testing	58
4.3.3.3.2.1	Authentication Controller (TC_WB_AUTH_001 - TC_WB_AUTH_017)	58
4.3.3.3.2.2	Book Controller (TC_WB_BOOK_001 - TC_WB_BOOK_008)	59
4.3.4	Non-Functional Testing	59
4.3.4.1	Performance Testing	59
4.3.4.2	Security Testing	63
4.3.4.3	Usability Testing	65
4.3.4.4	Browser Compatibility Testing	68
4.3.5	Regression Testing	71
4.3.6	Installation and Deployment Testing	73
4.3.7	Static Testing	75
4.3.7.1	Documentation Review	75
4.3.7.2	Code Review and Walkthroughs	75
4.3.7.3	Static Testing Test Cases and Findings	76
4.4	Test Environment and Tools	77
4.5	Evaluation and Reporting	78
5	Detailed Outcome of Each Testing Step	79
5.1	Summary of Test Execution	79
5.2	Unit and Component Testing Outcome	80
5.2.1	Backend Unit Testing	80
5.2.2	Frontend Unit & Component Testing	80
5.3	Integration Testing Outcome	80
5.4	System (End-to-End) Testing Outcome	81
5.5	White-Box Testing Outcome (Code Coverage)	81

5.5.1	Backend Code Coverage	81
5.5.2	Frontend Code Coverage	82
5.6	Non-Functional Testing Outcome	83
5.6.1	Performance Testing	83
5.6.2	Security Testing	83
5.6.3	Usability Testing	83
5.6.4	Browser Compatibility Testing	84
5.7	Regression and Deployment Testing Outcome	84
5.8	Static Testing Outcome	85
6	Conclusion	86
6.1	Summary of Testing Effort	86
6.2	Key Findings and Outcomes	87
6.3	Challenges and Lessons Learned	88
6.4	Future Recommendations	88
6.5	Final Reflection	89
	References	90
7	Members' Summary Report	91
7.1	Tishko Salah Hawez (Test Manager)	91
7.1.1	Overview of Role and Responsibilities	91
7.1.2	Test Planning and Strategy	91
7.1.3	Execution Management and Coordination	92
7.1.4	Reporting, Defect Management, and Quality Assurance	92
7.1.5	Documentation and Static Testing	93
7.1.6	Conclusion	94

List of Figures

1.1	High-Level System Architecture of the LMS.	8
2.1	High-Level System Architecture.	10
2.2	Conceptual Entity-Relationship Diagram (ERD).	11
3.1	Project Testing Schedule Gantt Chart	15
4.1	Overall Testing Process Flowchart	19
4.2	Jest Code Coverage Report	59
4.3	Librarian Dashboard UI for Usability Evaluation	68
4.4	Member Borrowal History UI for Usability Evaluation	68
4.5	Jest Test Execution Report	78
5.1	Backend Jest Test Execution Summary Report	79
5.2	Cypress E2E Test Execution Dashboard (Illustrative)	81
5.3	Backend Jest Code Coverage Report	82
5.4	Frontend Jest Code Coverage Report (Illustrative)	83
5.5	Browser Compatibility Check (Illustrative)	84
6.1	Frontend Jest Test Coverage Report	88
6.2	Backend Jest Test Coverage Report	88
6.3	Summary of Jest Code Coverage Reports	88
7.1	Frontend Test Coverage Report Overseen by the Test Manager.	93
7.2	Cypress Dashboard for E2E Test Run Monitoring.	93

List of Tables

3.1	Team Roles and Primary Responsibilities	14
3.2	Testing Tools and Frameworks	14
3.3	Risk Assessment and Mitigation Plan	16
4.1	Test Cases for authController	20
4.2	Test Cases for authorController	21
4.3	Test Cases for bookController	22
4.4	Test Cases for borrowalController	23
4.5	Test Cases for genreController	24
4.6	Test Cases for reviewController	25
4.7	Integration Test Cases	25
4.8	Authentication and Authorization Test Cases	30
4.9	Entity Management Test Cases	34
4.10	Use Case Test Cases	41
4.11	State Transition Test Cases	45
4.12	Specific Feature Test Cases	48
4.13	API Test Cases	52
4.15	Performance Test Cases	59
4.16	Security Test Cases	63
4.17	Usability Test Cases	66
4.18	Browser Compatibility Test Cases	69
4.19	Regression Test Suite	71
4.20	Installation & Deployment Test Cases	74
4.21	Static Testing Cases and Results	76
4.22	Testing Tools and Frameworks	77

Introduction

This report is submitted in fulfillment of the group coursework requirements for the SE507 Software Testing and Evaluation module. The primary objective of this coursework is to provide a practical, hands-on application of the fundamental principles and processes of software testing and evaluation. In accordance with the coursework guidelines, which permit the use of a ready-made application, this project undertakes a comprehensive testing and quality assurance initiative on a pre-existing software artifact: a full-stack Library Management System (LMS) sourced from the open-source repository MERNRAD on GitHub.

The work documented herein demonstrates a systematic approach to the entire testing life-cycle as applied to an existing codebase. By selecting a representative MERN-stack application, this project serves as a realistic case study in verifying software quality, ensuring functional correctness, and validating adherence to implicit and explicit requirements. The ultimate goal is to present an empirically grounded evaluation of the software's readiness for deployment, thereby bridging the theoretical concepts of software testing with their practical implementation in a real-world scenario.

1.1 Problem Domain and Project Rationale

The impetus for selecting this particular project stems from the universal problem domain it addresses: the operational inefficiencies inherent in manual or legacy library systems. The management of core library functions—such as cataloging, circulation, and user administration—presents significant logistical challenges that are amenable to technological solutions. As delineated in the system's features and our derived Software Requirements Specification (SRS), the LMS aims to provide a centralized, scalable, and efficient digital platform. It is designed to serve two distinct user cohorts: Librarians, who function as system administrators with privileged access, and Members, who are the primary consumers of the library's services.

The system's core functional domains, which form the basis of this testing investigation, include:

- **User and Access Control Management:** Implementation of secure authentication mechanisms and a role-based access control (RBAC) model to govern system access for Librarians and Members.
- **Entity Lifecycle Management:** Provision of comprehensive CRUD (Create, Read, Update, Delete) capabilities for foundational library entities, namely books, authors, and genres.
- **Circulation Management:** Automation of the complete borrowal lifecycle, from the issuance of materials to their return, including the monitoring of due dates and the maintenance of historical transaction records.

- **Interactive User Feedback System:** A feature enabling members to contribute qualitative and quantitative feedback through book reviews and ratings, fostering community engagement.

1.2 Purpose and Contribution of This Report

This document constitutes the definitive testing and evaluation report for the selected Library Management System. Its central purpose is to methodically detail the testing strategies, execution processes, and empirical outcomes derived from our quality assurance phase. The report endeavors to furnish verifiable evidence that the LMS satisfies its specified functional and non-functional requirements, thereby establishing a high degree of confidence in its deployment readiness.

The specific objectives of this report are to:

- Systematically document the entire testing lifecycle, from strategic planning and test case architecting through to execution, data logging, and results analysis.
- Present a transparent and detailed account of all executed test protocols, including quantified outcomes, identified anomalies, and defect classifications.
- Conduct a multi-dimensional evaluation of the software's quality attributes, encompassing functionality, performance, security, and usability.
- Furnish a conclusive, evidence-based assessment for stakeholders regarding the system's operational viability, in line with the course requirements.

1.3 Scope of the Testing Mandate

To ensure methodological rigor and comprehensive validation, a multi-layered testing strategy was devised and implemented. The scope of this mandate was designed to scrutinize the system at all levels of abstraction, from discrete software units to the integrated system as a whole. The investigation encompassed the following testing paradigms:

Static Testing: Critical analysis of project artifacts, including the SRS and source code, without dynamic execution, to identify defects, ambiguities, and deviations from coding standards at the earliest possible stage.

Unit Testing: Rigorous validation of discrete software units (e.g., functions, classes) within the frontend and backend codebases to verify their logical and functional correctness in isolation.

Integration Testing: Examination of the interoperability between system components, with a primary focus on the data exchange and control flow across the interface between the React frontend and the Node.js/Express backend API.

Functional Testing: Application of black-box testing methodologies to confirm that the system's observable features conform to the behaviors specified in the SRS, including use case, entity management, and state transition scenarios.

API Testing: Direct interrogation of the backend API endpoints to validate their adherence to the specified contract, including request handling, data processing, and response generation.

Non-Functional Testing: Assessment of critical system quality attributes beyond core functionality, including:

- **Performance Testing:** Empirical measurement of the system’s responsiveness, throughput, and stability under simulated load conditions.
- **Security Testing:** Proactive identification of potential security vulnerabilities to ensure data confidentiality, integrity, and availability.
- **Usability Testing:** Evaluation of the human-computer interaction (HCI) aspects of the user interface to gauge its effectiveness, efficiency, and user satisfaction.
- **Browser Compatibility Testing:** Verification of consistent rendering and functionality across a defined set of contemporary web browsers.

Regression Testing: Systematic re-validation of existing functionality to ensure that recent code modifications or defect resolutions have not introduced unintended side effects.

Installation and Deployment Testing: Confirmation that the application can be reliably built, containerized using Docker, and deployed to its target operational environment.

1.4 System Architecture Overview

The Library Management System is architected as a full-stack web application employing the MERN technology stack, a widely adopted paradigm for robust and scalable web service development.

- **MongoDB:** A NoSQL, document-oriented database serving as the persistence layer for all application data.
- **Express.js:** A minimalist backend framework for Node.js, providing the foundation for the RESTful API that encapsulates the system’s business logic.
- **React.js:** A declarative JavaScript library for building the dynamic, single-page application (SPA) that constitutes the client-side user interface.
- **Node.js:** A server-side JavaScript runtime environment that powers the application’s backend services.

Figure 2.1 provides a high-level schematic of the system’s architecture, illustrating the fundamental interactions between the client, server, and database tiers.

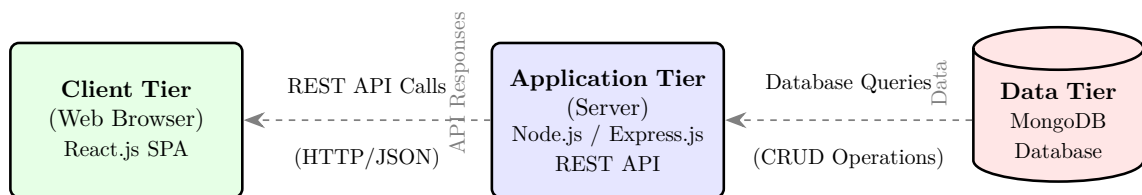


Figure 1.1: High-Level System Architecture of the LMS.

Background

This chapter provides a comprehensive overview of the Library Management System, the System Under Test (SUT). It details the system's purpose, architecture, user roles, core functionalities, and underlying data model. This context is essential for understanding the scope, strategy, and execution of the testing activities detailed in subsequent chapters.

2.1 System Under Test (SUT) Overview

The Library Management System is a full-stack web application designed to modernize and streamline library operations **srs**. Its primary purpose is to provide a digital platform for managing library resources, including books, authors, and genres, while also managing user accounts for both librarians and library members **srs**. The system aims to replace or supplement traditional, manual library management methods with an efficient, accessible, and user-friendly web-based solution **srs**.

The project's scope encompasses a complete, self-contained system featuring a backend API, a database, and a frontend client application **srs**. Key capabilities include:

- **User Authentication:** Secure user registration, login, and logout functionalities with role-based access control for two distinct user types: Librarians and Members **srs**.
- **Inventory Management:** Comprehensive CRUD (Create, Read, Update, Delete) operations for the core library entities: books, authors, and genres **srs**.
- **Borrowal Tracking:** Management of book borrowals and returns, allowing librarians to oversee lending activities and members to view their history **srs**.
- **User Administration:** Tools for librarians to manage all user accounts within the system **srs**.
- **Review System:** A newly introduced feature allowing members to submit reviews for books, which can then be managed by librarians **srs**.

2.2 System Architecture

The system is built on a modern, multi-tier architecture, which separates concerns between the client, server, and database. This design promotes modularity, scalability, and maintainability.

- **Frontend (Client-Side):** The user interface is a single-page application (SPA) developed using **React** and the **Material-UI (MUI)** component library. It is responsible for rendering the UI and interacting with the backend via API calls. It runs in any modern web browser **srs**.

- **Backend (Server-Side):** The backend is a RESTful API built with **Node.js** and the **Express.js** framework. It handles all business logic, data processing, and user authentication using Passport.js **srs**.
- **Database:** Data persistence is managed by a **MongoDB** database, a NoSQL database that stores data in flexible, JSON-like documents. The backend interacts with MongoDB via the Mongoose ODM library **srs**.

The entire application is containerized using **Docker**, ensuring consistent development, testing, and deployment environments **srs**. Figure 2.1 illustrates the high-level architecture.

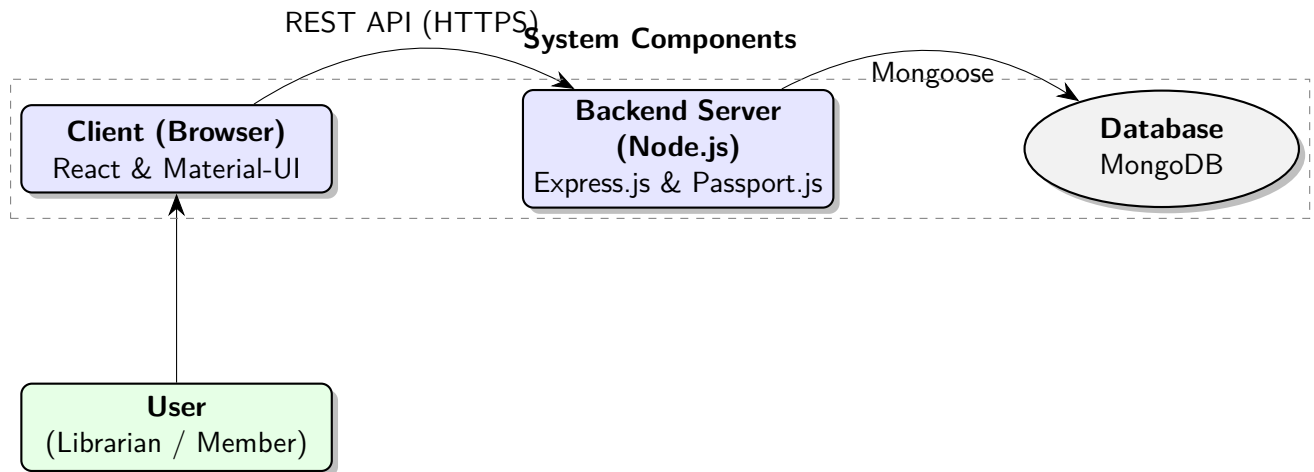


Figure 2.1: High-Level System Architecture.

2.3 User Roles and Functionalities

The system defines two distinct user roles, each with a specific set of permissions and responsibilities as outlined in the SRS **srs**.

2.3.1 Librarian (Admin)

Librarians are the administrators of the system with full control over all its aspects. Their primary responsibilities include:

- Managing the library's entire inventory, including adding, updating, and deleting books, authors, and genres **srs**.
- Overseeing all user accounts, which involves creating new accounts for both members and other librarians, as well as updating and deleting existing accounts **srs**.
- Managing the complete lifecycle of book borrowals, from creation to marking returns **srs**.
- Viewing system-wide statistics and data on a dedicated dashboard **srs**.
- Moderating book reviews submitted by members **srs**.

2.3.2 Member

Members are the standard users of the library. Their access is generally limited to viewing data and managing their own activities. Their functions include:

- Browse the library's catalog of books, authors, and genres **srs**.

- Requesting to borrow available books **srs**.
- Viewing their personal borrowing history **srs**.
- Submitting, viewing, and managing their own reviews for books **srs**.

2.4 Data Model

The system's data is structured around several core entities, whose relationships define the application's functionality. The main entities are **User**, **Book**, **Author**, **Genre**, **Borrowal**, and **Review srs**. The relationships between these entities are a key focus of integration testing to ensure data integrity. For instance, a *Borrowal* record links a *User* to a *Book*, and a *Book* can have one *Author* and one *Genre srs*.

Figure 2.2 provides a conceptual Entity-Relationship Diagram (ERD) illustrating these connections.

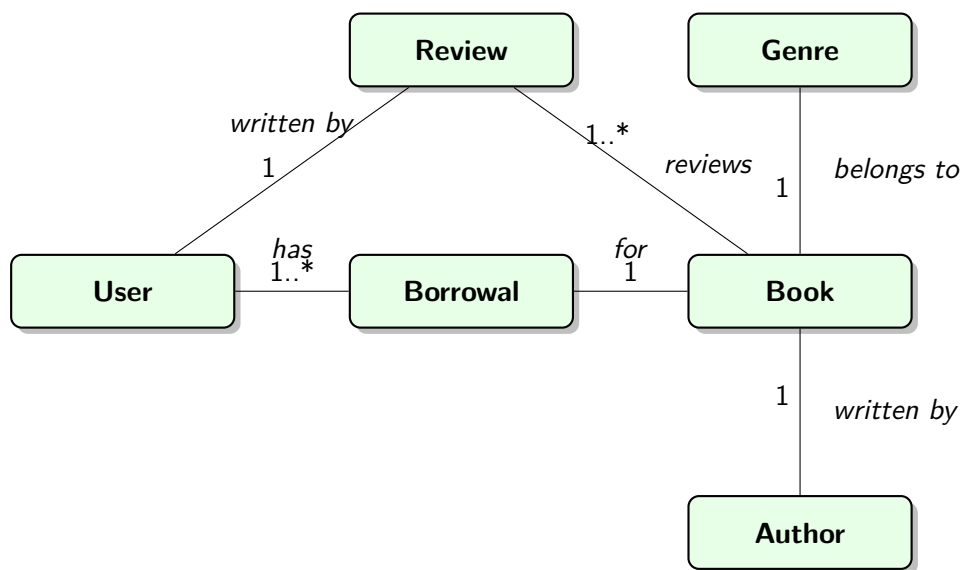


Figure 2.2: Conceptual Entity-Relationship Diagram (ERD).

Test Management

This section outlines the comprehensive management plan for the testing of the Library Management System. The plan is designed to ensure that all software components are rigorously tested, meet the requirements specified in the Software Requirements Specification (SRS), and achieve the highest standards of quality. This document details the test strategy, scope, team roles, environment, schedule, deliverables, risk management, and success criteria for the project.

3.1 Test Strategy

Our test strategy employs a multi-layered approach, integrating various testing levels to ensure comprehensive coverage and early defect detection. The strategy is aligned with the V-Model, ensuring that testing activities are planned in parallel with development phases.

3.1.1 Overall Approach

The testing process is structured across multiple levels:

- **Static Testing:** Performed without executing the code, focusing on reviews and analysis of documentation and source code to find defects early.
- **Unit Testing:** Developers will test individual components (functions, classes) in isolation to verify their correctness.
- **Integration Testing:** Focuses on testing the interfaces and interactions between integrated components, such as the communication between the frontend client and the backend API.
- **System & E2E Testing:** The complete, integrated system is tested from end-to-end to validate that it meets all specified requirements from a user's perspective.
- **Non-Functional Testing:** Ensures the system meets requirements for performance, security, usability, and compatibility.
- **Regression Testing:** Conducted after modifications to ensure that new changes have not introduced new defects into existing functionality.

3.1.2 Test Levels

- **Unit Testing:** Implemented using **Jest**. For the backend, this involves testing Node.js controllers, models, and utility functions. For the frontend, this includes testing React components, hooks, and utility functions using Jest and the React Testing Library.

- **API & Integration Testing:** Backend API endpoints are tested using **Jest** and **Supertest** to verify CRUD operations, authentication flows, and business logic. This also covers the integration points between different services.
- **End-to-End (E2E) Testing:** Real user workflows are simulated using **Cypress**. These tests cover complete user journeys, such as logging in, searching for a book, borrowing it, and logging out.
- **Non-Functional Testing:**
 - **Performance Testing:** **K6** is used to script and execute load tests against the API to measure response times, throughput, and stability under load.
 - **Security Testing:** Manual and automated checks are performed to identify common vulnerabilities (e.g., SQL injection, XSS, insecure direct object references). Cypress tests are used to check for authorization flaws.
 - **Usability & Compatibility Testing:** Manual testing and Cypress scripts ensure the application is user-friendly and renders correctly across major web browsers (Chrome, Firefox, Safari).

3.2 Test Scope

3.2.1 In Scope

The following features and functionalities, as defined in the SRS, are within the scope of testing:

- User Authentication (Login, Logout, Role-based Access Control)
- Librarian Dashboard and statistical widgets
- Full CRUD (Create, Read, Update, Delete) operations for Books, Authors, and Genres
- Borrowal Management (borrowing, returning, and tracking books)
- User Management (Librarian managing member accounts)
- Book Review and Rating system
- API endpoint functionality and security
- Database referential integrity

3.2.2 Out of Scope

The following are considered out of scope for testing:

- Testing of third-party libraries and frameworks (e.g., React, Express, Mongoose).
- Performance of the underlying hardware, network infrastructure, or MongoDB database engine itself.
- Exhaustive testing of every possible combination of user inputs.
- Usability testing on mobile operating systems (unless specified).

3.3 Team Roles and Responsibilities

The testing process involves the entire team, with specific roles and responsibilities as defined in the project's task distribution plan.

Table 3.1: Team Roles and Primary Responsibilities

Member	Role	Primary Testing Responsibilities
Tishko Salah	Test Manager	Oversees the entire testing process, manages the test plan and schedule, coordinates team efforts, and is the final authority on test completion.
Sara Zuhair	Code Specifier	Focuses on non-coder testing activities, including functional validation against SRS, documentation review, and usability testing. Leads the execution of user-centric test cases.
Yassin Hussein	Code Reviewer	Leads technical testing activities, including unit testing, API testing, and code reviews. Ensures code quality, performance, and adherence to standards.
Ravyar Barzan	Document Inspector	Leads the review and inspection of all project documentation (SRS, Test Plan, Test Cases, Final Report) for clarity, consistency, and completeness.

3.4 Test Environment and Tools

All testing is conducted within containerized environments managed by Docker to ensure consistency and reproducibility.

- **Operating Systems:** Windows, macOS, and Linux (for development and execution).
- **Test Environments:** Defined in `docker-compose.dev.yml`, `docker-compose.test.yml`, and `docker-compose.prod.yml` to simulate development, testing, and production setups.
- **Version Control:** Git, with GitHub for repository hosting and collaboration.

Table 3.2: Testing Tools and Frameworks

Category	Tool/Framework	Purpose
Backend Testing	Node.js, Jest, Supertest	Unit and integration testing of API endpoints and services.
Frontend Testing	Jest, React Testing Library	Unit and component testing of React components and logic.
E2E Testing	Cypress	Automated end-to-end testing of user workflows in a browser.
Performance	K6, Docker	Load testing the backend API to measure performance metrics.
Code Quality	ESLint, Prettier	Static code analysis to enforce coding standards and find bugs.
Documentation	LaTeX, Draw.io	Creating project reports, diagrams, and test artifacts.

3.5 Test Schedule

The project timeline spans approximately 14 weeks, from early March to mid-June 2025. The schedule is visualized in the Gantt chart below (Figure 3.1).

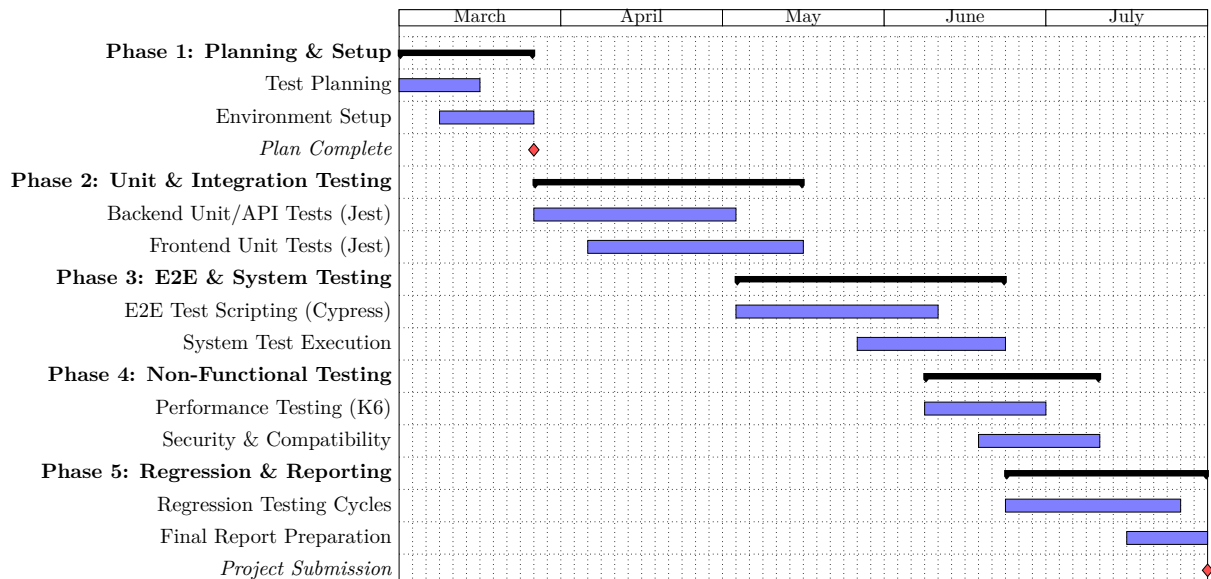


Figure 3.1: Project Testing Schedule Gantt Chart

3.6 Deliverables

The following artifacts will be produced as part of the testing process:

- **Test Plan:** This comprehensive document (of which this chapter is a part).
- **Test Case Documentation:** Detailed test cases for all levels of testing, located in `/docs/Report/test_artifacts/test_cases/`.
- **Test Scripts:** All automated test scripts written using Jest, Cypress, and K6.
- **Defect Reports:** Bugs and issues will be tracked, prioritized, and managed. (A system like GitHub Issues will be used).
- **Test Execution Reports:** Automated reports generated by Jest (`/server/test-report/report.html`) and Cypress (videos/screenshots) after test runs.
- **Final Test Summary Report:** The complete final report detailing all testing activities, results, and conclusions.

3.7 Risk Management

Potential risks that could impact the testing schedule and quality are identified and managed proactively.

Table 3.3: Risk Assessment and Mitigation Plan

Risk Description	Likelihood	Impact	Mitigation Strategy
Delays in development phases impact testing start dates.	Medium	High	Maintain close communication between developers and testers. Adapt the test plan and prioritize testing of critical features.
Test environment is unstable or differs from production.	Low	High	Use Docker to ensure consistent environments. Perform smoke tests on deployment to validate environment stability.
Critical defects are discovered late in the project timeline.	Medium	High	Emphasize early testing (unit, integration). Prioritize test cases based on risk and critical user workflows.
Team member unavailability due to unforeseen circumstances.	Low	Medium	Ensure knowledge is shared across the team. Cross-train members on critical testing tasks and tools.
Underestimation of time required for test case creation or execution.	Medium	Medium	Base estimates on feature complexity. Regularly review progress and re-allocate resources as needed.

3.8 Entry and Exit Criteria

Clear criteria define the start and end points for the formal testing process.

3.8.1 Entry Criteria

Testing on a feature or build can begin when:

- The SRS and relevant design documents are complete and approved.
- The code has been developed, peer-reviewed, and successfully built.
- All developer-written unit tests for the feature are passing.
- The required test environment and test data are available and stable.
- Test cases have been written, reviewed, and approved by the test manager.

3.8.2 Exit Criteria

The overall testing phase will be considered complete when:

- All planned test cases have been executed.
- A pass rate of 100% for all critical and major test cases is achieved.
- A pass rate of over 95% for minor test cases is achieved.
- No outstanding Severity-1 (blocker) or Severity-2 (critical) defects remain open.

- All major functional and non-functional requirements are verified and met.
- The final test report is completed and approved by all team members.

Testing and Evaluation Process

This section details the comprehensive testing and evaluation process designed to ensure the Library Management System (LMS) meets the specified requirements and quality standards. Our strategy employs a multi-layered approach, combining static and dynamic testing techniques across various levels of the application architecture. The process is structured to systematically identify, document, and resolve defects throughout the software development lifecycle (SDLC).

The primary objectives of our testing process are to:

- Verify that all functional requirements outlined in the Software Requirements Specification (SRS) are correctly implemented.
- Ensure that the system meets all non-functional requirements, including performance, security, and usability benchmarks.
- Validate the system's stability, reliability, and robustness through rigorous integration, regression, and end-to-end testing.
- Provide comprehensive test coverage reports and defect analyses to stakeholders for informed decision-making.

4.1 Testing Strategy Overview

Our testing strategy is built upon a foundation of continuous testing, integrating quality assurance activities into every phase of the development cycle. The strategy is visualized in Figure 4.1. It begins with static testing techniques like code and documentation reviews, followed by a hierarchy of dynamic testing levels, from unit testing of individual components to system-wide end-to-end testing.

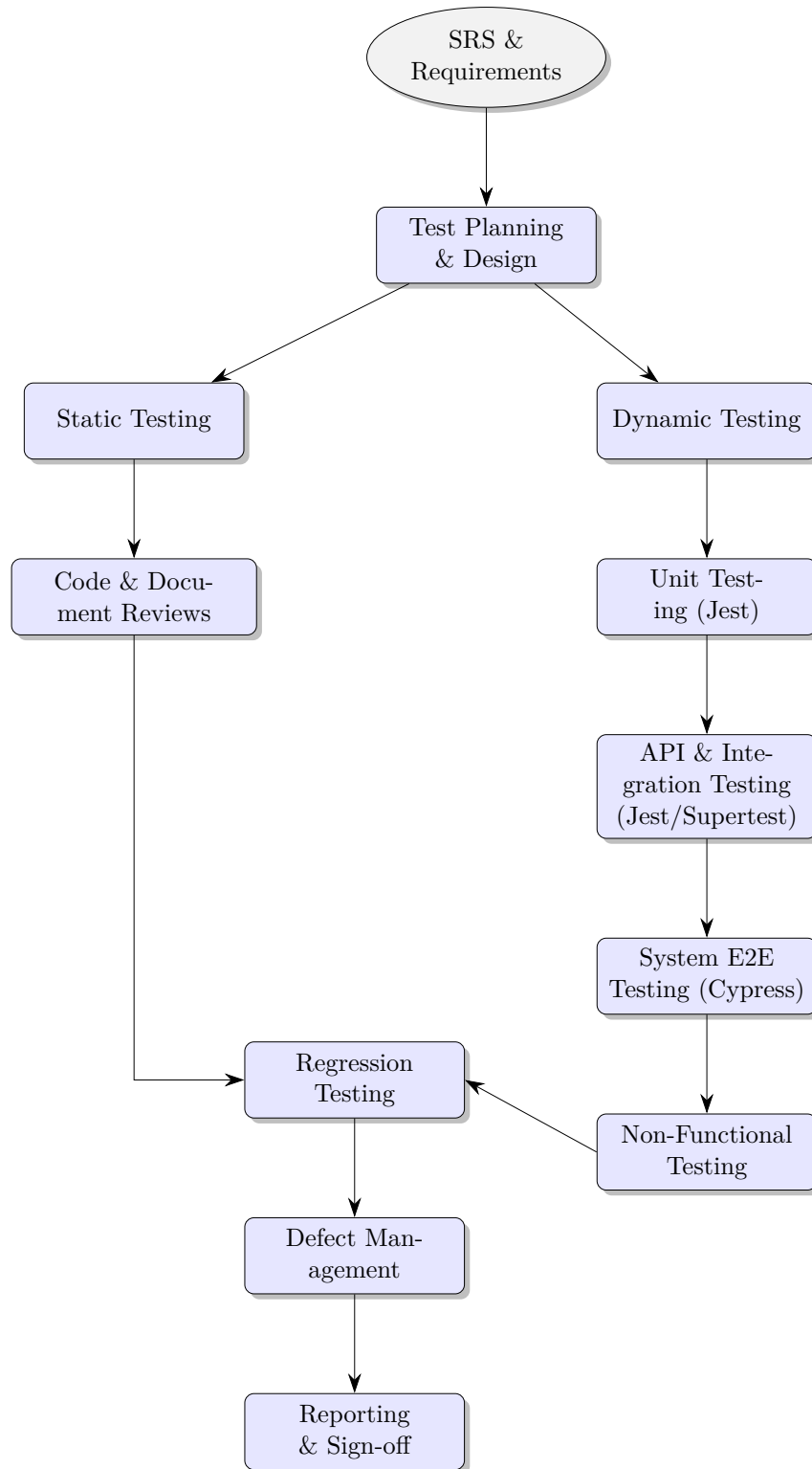


Figure 4.1: Overall Testing Process Flowchart

4.2 Testing Levels

Testing is structured into distinct levels to ensure all parts of the system are validated, from the smallest code unit to the fully integrated application.

4.2.1 Unit Testing

Unit tests focus on the smallest individual components of the application in isolation. This is primarily aimed at the controllers, models, and utility functions within the Node.js backend.

- **Objective:** To verify that each function or module behaves as expected.
- **Framework:** Jest is used as the testing framework, assertion library, and test runner.
- **Methodology:** Dependencies such as database models or external services are mocked to ensure tests are fast, reliable, and isolated. Test cases cover both successful paths and error conditions.

Detailed unit test cases are documented in the corresponding test file.

4.2.1.1 Authentication Controller (authController)

The Auth Controller is responsible for user authentication, including registration (by a librarian), login, and logout. These tests ensure that access control and user session management are functioning correctly, as specified in the SRS.

Table 4.1: Test Cases for authController

Test Case ID	Related Requirement (SRS)	Test Scenario	Test Steps	Expected Result
TC-AUTH-001	3.1.1 User Registration , UC-004	Successful user registration	1. Mock <code>User.findOne</code> to resolve as null. 2. Mock <code>User.create</code> to resolve with a new user object. 3. Call <code>registerUser</code> with valid new user data.	1. Response status is 201. 2. Response JSON indicates successful registration.
TC-AUTH-002	UC-004 (A1: Email Already Exists)	Attempt to register a user that already exists	1. Mock <code>User.findOne</code> to resolve with an existing user object. 2. Call <code>registerUser</code> with data of an existing user.	1. Response status is 400. 2. Response JSON indicates that the username or email already exists.

Continued on next page

Table 4.1 – continued from previous page

Test Case ID	Related Requirement (SRS)	Test Scenario	Test Steps	Expected Result
TC-AUTH-003	3.1.2 User Login , UC-001	Successful user login with valid credentials	1. Mock <code>passport.authenticate</code> to succeed and return a user object. 2. Call <code>loginUser</code> middleware with correct username and password.	1. <code>req.logIn</code> is called. 2. Response status is 200. 3. Response JSON indicates successful login and includes user data.
TC-AUTH-004	UC-001 (A1: Invalid Credentials)	Attempt to login with invalid credentials	1. Mock <code>passport.authenticate</code> to fail (returns false). 2. Call <code>loginUser</code> middleware with an incorrect username or password.	1. Response status is 401. 2. Response JSON indicates incorrect credentials.
TC-AUTH-005	3.1.3 User Logout	Successful user logout	1. Mock <code>req.logout</code> . 2. Call <code>logoutUser</code> .	1. <code>req.logout</code> is called. 2. Response status is 200. 3. Response JSON indicates successful logout.

4.2.1.2 Author Controller (authorController)

The Author Controller manages CRUD operations for author entities. These tests validate the creation and retrieval of authors.

Table 4.2: Test Cases for authorController

Test Case ID	Related Requirement (SRS)	Test Scenario	Test Steps	Expected Result
TC-ATHR-001	3.4 Author Management (POST /api/author/add)	Successfully create a new author	1. Provide author details in <code>req.body</code> . 2. Mock <code>Author.create</code> to resolve with the new author data. 3. Call <code>createAuthor</code> .	1. Response status is 201. 2. Response JSON contains the newly created author.
TC-ATHR-002	3.4 Author Management (GET /api/author/getAll)	Successfully retrieve all authors	1. Mock <code>Author.find</code> to resolve with an array of author objects. 2. Call <code>getAllAuthors</code> .	1. Response status is 200. 2. Response JSON contains the array of all authors.

4.2.1.3 Book Controller (bookController)

The Book Controller handles all CRUD operations for book entities. The tests ensure that books can be created, retrieved (all and by ID), updated, and deleted as per the requirements.

Table 4.3: Test Cases for bookController

Test Case ID	Related Requirement (SRS)	Test Scenario	Test Steps	Expected Result
TC-BOOK-001	3.3 Book Management (POST /api/book/add) , UC-002	Successfully create a new book	1. Provide book details in <code>req.body</code> . 2. Mock <code>Book.create</code> to resolve successfully. 3. Call <code>createBook</code> .	1. Response status is 201. 2. Response JSON contains the new book.
TC-BOOK-002	UC-002 (A2: API Error)	Fail to create a book due to a database error	1. Mock <code>Book.create</code> to reject with an error. 2. Call <code>createBook</code> .	1. Response status is 500. 2. Response JSON contains an error message.
TC-BOOK-003	3.3 Book Management (GET /api/-book/getAll)	Successfully retrieve all books	1. Mock <code>Book.find().populate()</code> to resolve with an array of books. 2. Call <code>getAllBooks</code> .	1. Response status is 200. 2. Response JSON contains the array of books.
TC-BOOK-004	3.3 Book Management (GET /api/-book/get/:id)	Successfully retrieve a single book by its ID	1. Provide a book ID in <code>req.params</code> . 2. Mock <code>Book.findById().populate()</code> to resolve with a book object. 3. Call <code>getBookById</code> .	1. Response status is 200. 2. Response JSON contains the specific book.
TC-BOOK-005	3.3 Book Management (GET /api/-book/get/:id)	Attempt to retrieve a book with a non-existent ID	1. Provide a non-existent book ID in <code>req.params</code> . 2. Mock <code>Book.findById().populate()</code> to resolve as null. 3. Call <code>getBookById</code> .	1. Response status is 404. 2. Response JSON contains a 'Book not found' message.
TC-BOOK-006	3.3 Book Management (PUT /api/-book/update/:id)	Successfully update an existing book	1. Provide a book ID and update data. 2. Mock <code>Book.findByIdAndUpdate</code> to resolve with the updated book. 3. Call <code>updateBook</code> .	1. Response status is 200. 2. Response JSON contains the updated book data.

Continued on next page

Table 4.3 – continued from previous page

Test Case ID	Related Requirement (SRS)	Test Scenario	Test Steps	Expected Result
TC-BOOK-007	3.3 Book Management (DELETE /api/-book/delete/:id)	Successfully delete a book	1. Provide a book ID to be deleted. 2. Mock <code>Book.findByIdAndDelete</code> to resolve successfully. 3. Call <code>deleteBook</code> .	1. Response status is 200. 2. Response JSON contains a 'Book removed' message.

4.2.1.4 Borrowal Controller (borrowalController)

This controller manages borrowal records. Tests cover creating a borrowal (and updating book availability), updating a borrowal's status, and retrieving borrowals based on user roles (Librarian vs. Member).

Table 4.4: Test Cases for borrowalController

Test Case ID	Related Requirement (SRS)	Test Scenario	Test Steps	Expected Result
TC-BORR-001	3.6 Borrowal Management , UC-003	Create a borrowal for an available book	1. Mock <code>Book.findById</code> to return an available book. 2. Mock <code>Borrowal.create</code> to succeed. 3. Call <code>createBorrowal</code> .	1. Book's availability is set to false. 2. Response status is 201. 3. Response JSON contains the new borrowal record.
TC-BORR-002	UC-003 (A1: Book Not Available)	Attempt to create a borrowal for an unavailable book	1. Mock <code>Book.findById</code> to return an unavailable book. 2. Call <code>createBorrowal</code> .	1. Response status is 400. 2. Response JSON has a "Book is not available" message.
TC-BORR-003	3.6 Borrowal Management (PUT /api/borrowal/update/:id)	Update borrowal status to 'Returned'	1. Mock <code>Borrowal.findById</code> to find a borrowal. 2. Mock <code>Book.findByIdAndUpdate</code> to succeed. 3. Call <code>updateBorrowal</code> with status 'Returned'.	1. Book's availability is set to true. 2. Response status is 200. 3. Response JSON contains the updated borrowal.
Continued on next page				

Table 4.4 – continued from previous page

Test Case ID	Related Requirement (SRS)	Test Scenario	Test Steps	Expected Result
TC-BORR-004	3.6 Borrowal Management (GET /api/borrowal/getAll)	Get all borrowals as a Librarian	1. Set <code>req.user.role</code> to 'Librarian'. 2. Mock <code>Borrowal.find</code> to return all borrowals. 3. Call <code>getBorrowals</code>.	1. <code>Borrowal.find</code> is called with an empty filter. 2. Response contains all borrowal records.
TC-BORR-005	3.6 Borrowal Management , UC-005	Get own borrowals as a Member	1. Set <code>req.user.role</code> to 'Member'. 2. Mock <code>Borrowal.find</code> to return member-specific borrowals. 3. Call <code>getBorrowals</code>.	1. <code>Borrowal.find</code> is called with a filter for the member's ID. 2. Response contains only the member's borrowals.

4.2.1.5 Genre Controller (`genreController`)

The Genre Controller manages CRUD operations for genre entities. These tests validate the creation and retrieval of genres.

Table 4.5: Test Cases for `genreController`

Test Case ID	Related Requirement (SRS)	Test Scenario	Test Steps	Expected Result
TC-GENR-001	3.5 Genre Management (POST /api/genre/add)	Successfully create a new genre	1. Provide genre details in <code>req.body</code>. 2. Mock <code>Genre.create</code> to resolve with the new genre data. 3. Call <code>createGenre</code>.	1. Response status is 201. 2. Response JSON contains the newly created genre.
TC-GENR-002	3.5 Genre Management (GET /api/genre/getAll)	Successfully retrieve all genres	1. Mock <code>Genre.find</code> to resolve with an array of genre objects. 2. Call <code>getAllGenres</code>.	1. Response status is 200. 2. Response JSON contains the array of all genres.

4.2.1.6 Review Controller (`reviewController`)

The Review Controller is responsible for managing book reviews. The tests cover the creation of reviews and fetching all reviews for a specific book.

Table 4.6: Test Cases for reviewController

Test Case ID	Related Requirement (SRS)	Test Scenario	Test Steps	Expected Result
TC-REV-001	3.8 Review Management (POST /api/review/add)	Successfully create a review for a book	1. Provide review details (bookId, rating, comment) in req.body. 2. Mock <code>Review.create</code> to resolve with the new review object. 3. Call <code>createReview</code>.	1. Response status is 201. 2. Response JSON contains the newly created review.
TC-REV-002	3.8 Review Management	Get all reviews for a specific book	1. Provide a book ID in <code>req.params.bookId</code>. 2. Mock <code>Review.find().populate()</code> to resolve with an array of reviews. 3. Call <code>getReviewsForBook</code>.	1. <code>Review.find</code> is called with a filter for the book ID. 2. Response status is 200. 3. Response JSON contains the array of reviews for that book.

4.2.2 Integration Testing

Integration testing focuses on verifying the interaction between different components. For the LMS, this primarily involves testing the API endpoints to ensure they correctly interact with controllers, models, and the database.

- **Objective:** To detect defects in the interfaces and interactions between integrated components.
- **Framework:** Jest and Supertest are used to send HTTP requests to the API endpoints and assert the responses.
- **Methodology:** Tests cover creating, reading, updating, and deleting (CRUD) entities, as well as complex workflows involving multiple API calls. A test database is used to ensure a clean state for each test run.

The specific test cases for integration testing are detailed in:

Table 4.7: Integration Test Cases

Test Case ID	Test Title / Scenario	Test Steps	Expected Results
Scenario: Full Lifecycle - From Entity Creation to Borrowal			
Continued on next page			

Table 4.7 – continued from previous page

Test Case ID	Test Title / Scenario	Test Steps	Expected Results
TC_INT_001	End-to-end workflow of creating all required entities for a new book, registering a new member, and the member borrowing that book.	• As Librarian: Login to the system. • Navigate to Author Management and add a new author (e.g., "J.R.R. Tolkien"). • Navigate to Genre Management and add a new genre (e.g., "Fantasy"). • Navigate to Book Management and add a new book ("The Hobbit"), linking the newly created author and genre. Set 'isAvailable' to true. • Navigate to User Management and register a new user with 'isAdmin=false' (e.g., "Member1"). • Logout. • As Member1: Login to the system. • Navigate to the book list and find "The Hobbit". • Initiate and confirm the borrowing of "The Hobbit".	• The author is created successfully and selectable in the book form. • The genre is created successfully and selectable in the book form. • The book is created with correct author and genre associations. The book list shows its status as available. • The new member account is created and appears in the user list. • Member1 can log in successfully and is directed to the member landing page. • A new borrowal record is created for Member1 and "The Hobbit". • The 'isAvailable' status of "The Hobbit" in the Book entity is updated to 'false'. • The new borrowal record appears in Member1's borrowal history.
Scenario: Data Integrity and Referential Constraints			
Continued on next page			

Table 4.7 – continued from previous page

Test Case ID	Test Title / Scenario	Test Steps	Expected Results
TC_INT_002	Verify data integrity when a linked entity (Author) is deleted after being associated with another entity (Book).	• As Librarian: Login. • Create a new, unique Author (e.g., "IntegTest Author"). • Create a new Book (e.g., "IntegTest Book") and link it to "IntegTest Author". • Verify the book details page shows "IntegTest Author" as the author. • Navigate to Author Management and delete "IntegTest Author". • Navigate back to Book Management and view the details of "IntegTest Book".	• The Author is successfully deleted. • The Book "IntegTest Book" still exists in the system. • The 'authorId' field for "IntegTest Book" is handled gracefully (e.g., set to null or an empty value), as the data model indicates it's optional. The book details page no longer displays the deleted author's name. • The system does not crash and remains in a consistent state.
TC_INT_003	Verify that deleting a User with active Borrowals does not violate data consistency.	• As Librarian: Login. • Ensure a specific Member (e.g., "MemberToDelete") has at least one active borrowal record. If not, create one. • Navigate to User Management. • Attempt to delete the user "MemberToDelete".	• Expected (Option A - Deletion Blocked): The system prevents the deletion and displays an error message indicating the user has active borrowals that must be resolved first. • Expected (Option B - Soft Delete/Orphaning): The user is deleted. The 'memberId' in the Borrowal record remains but now points to a non-existent user. The borrowal record is still visible to the Librarian, but may indicate the user is deleted. • (The exact expected result depends on the implemented business rule for referential integrity, which should be consistent).
Scenario: State Transitions Across Modules			
Continued on next page			

Table 4.7 – continued from previous page

Test Case ID	Test Title / Scenario	Test Steps	Expected Results
TC_INT_004	Verify that the state of a Book (isAvailable) is correctly updated by actions in the Borrowal module.	• Prerequisite: A book "StateTest Book" exists and is available. A "StateTest Member" exists. • As Member: Login as "StateTest Member". Borrow "StateTest Book". • As Librarian: Login. Navigate to Book Management. View the status of "StateTest Book". • Try to create a new borrowal for "StateTest Book" for another member. • Navigate to Borrowal Management. Find the borrowal record for "StateTest Book" and update its status to "Returned". • Navigate back to Book Management and view the status of "StateTest Book" again.	• After the member borrows the book, the book's 'isAvailable' status immediately changes to 'false'. • The Librarian cannot create a new borrowal for the now-unavailable book. The UI should prevent this action. • After the Librarian marks the borrowal as "Returned", the borrowal record's status field is updated. • The 'isAvailable' status of "StateTest Book" in the Book module reverts to 'true'. • The data remains consistent between the Book and Borrowal modules throughout the workflow.
Scenario: Role-Based Access Control (RBAC) Integration			
Continued on next page			

Table 4.7 – continued from previous page

Test Case ID	Test Title / Scenario	Test Steps	Expected Results
TC_INT_005	Verify that user roles created in the User module correctly restrict access to features in other modules (e.g., Book, Author).	• As Librarian: Login. Create a new user "TestMember" with the role of Member ('isAdmin=false'). • Logout. • As Member: Login as "TestMember". • Attempt to access the User Management page via direct URL navigation (e.g., '/users'). • Navigate to the Books list. • Check for the presence of "Add New Book", "Edit", or "Delete" buttons/options. • Using an API client (e.g., Postman), attempt to send a 'POST' request to '/api/book/add' using the session/token of "TestMember".	• The Member login is successful. • Direct navigation to the '/users' URL is blocked, and the user is redirected to the member dashboard or an error page. • The UI for the Books list does not show any administrative controls (Add, Edit, Delete). • The API request to 'POST /api/book/add' fails with an authorization error (e.g., 403 Forbidden or 401 Unauthorized), demonstrating that the access control middleware correctly uses the user's role from the session.

4.2.3 System Testing

System testing evaluates the complete and fully integrated application from an end-to-end (E2E) perspective. This level of testing simulates real user scenarios to validate the system against the requirements defined in the SRS.

- **Objective:** To validate the complete system's compliance with its specified requirements.
- **Framework:** Cypress is used for E2E testing, automating user interactions in a real browser environment.
- **Methodology:** Test scripts simulate user workflows such as logging in, searching for a book, borrowing a book, and managing user accounts. Assertions are made on the UI to confirm that the application behaves as expected.

4.3 Dynamic Testing Types

Dynamic testing is performed while the code is executing. We have categorized our dynamic tests into functional and non-functional types to ensure comprehensive coverage.

4.3.1 Functional Testing

Functional testing verifies that the system performs its functions as specified in the SRS. Test cases are derived directly from the user stories and functional requirements.

4.3.1.1 Authentication and Authorization

Table 4.8: Authentication and Authorization Test Cases

Test Case ID	Test Title	Test Steps	Expected Results
Sub-Section: User Registration (by Librarian)			
TC_AUTH_REG_001	Successful new user (Member) registration by Librarian	• Login as Librarian. • Navigate to 'User Management' or 'Add User' section. • Fill in all required fields with valid and unique data for a new 'Member' (e.g., email, password, full name). • Submit the registration form.	• System accepts the data. • A success message "User registered successfully" (or similar) is displayed. • The new user appears in the list of users. • The new user can subsequently log in (to be tested in Login cases).
TC_AUTH_REG_002	Attempt to register a new user with an existing email	• Login as Librarian. • Navigate to 'Add User' section. • Fill in user details, using an email address that already exists in the system. • Submit the registration form.	• System rejects the registration. • An appropriate error message is displayed (e.g., "Email already exists"). • The user is not created.
TC_AUTH_REG_003	Attempt to register a new user with missing required fields	• Login as Librarian. • Navigate to 'Add User' section. • Fill in user details but leave one or more mandatory fields blank (e.g., password or email). • Submit the registration form.	• System rejects the registration. • An appropriate error message is displayed, indicating which fields are required. • The user is not created.
Continued on next page			

Table 4.8 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_AUTH_REG_004	Attempt to register a new user with invalid data format (e.g., email)	• Login as Librarian. • Navigate to 'Add User' section. • Fill in user details, providing an incorrectly formatted email address (e.g., "usertest.com"). • Submit the registration form.	• System rejects the registration. • An appropriate error message is displayed (e.g., "Invalid email format"). • The user is not created.
Sub-Section: User Login			
TC_AUTH_LOGIN_001	Successful login with valid Librarian credentials	• Navigate to the login page. • Enter valid Librarian email. • Enter corresponding valid password. • Click the 'Login' button.	• User is authenticated successfully. • User is redirected to the Librarian dashboard/homepage. • Librarian-specific functionalities are visible/accessible.
TC_AUTH_LOGIN_002	Successful login with valid Member credentials	• Navigate to the login page. • Enter valid Member email. • Enter corresponding valid password. • Click the 'Login' button.	• User is authenticated successfully. • User is redirected to the Member dashboard/homepage. • Member-specific functionalities are visible/accessible.
TC_AUTH_LOGIN_003	Attempt login with invalid email	• Navigate to the login page. • Enter an email that does not exist. • Enter any password. • Click the 'Login' button.	• Login fails. • An appropriate error message is displayed (e.g., "Invalid email or password"). • User remains on the login page.
Continued on next page			

Table 4.8 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_AUTH_LOGIN_004	Attempt login with valid email but invalid password	• Navigate to the login page. • Enter a valid existing email. • Enter an incorrect password for that email. • Click the 'Login' button.	• Login fails. • An appropriate error message is displayed (e.g., "Invalid email or password"). • User remains on the login page.
TC_AUTH_LOGIN_005	Attempt login with empty email field	• Navigate to the login page. • Leave the email field empty. • Enter any password. • Click the 'Login' button.	• Login fails. • An appropriate error message is displayed (e.g., "Email is required"). • User remains on the login page.
TC_AUTH_LOGIN_006	Attempt login with empty password field	• Navigate to the login page. • Enter a valid email. • Leave the password field empty. • Click the 'Login' button.	• Login fails. • An appropriate error message is displayed (e.g., "Password is required"). • User remains on the login page.
Sub-Section: User Logout			
TC_AUTH_LOGOUT_001	Successful logout for a logged-in Librarian	• Login as Librarian. • Click the 'Logout' button/link.	• User is logged out successfully. • User is redirected to the login page or public homepage. • Session is terminated (user cannot access protected pages without logging in again).
TC_AUTH_LOGOUT_002	Successful logout for a logged-in Member	• Login as Member. • Click the 'Logout' button/link.	• User is logged out successfully. • User is redirected to the login page or public homepage. • Session is terminated.
Sub-Section: Role-Based Access Control (RBAC)			
Continued on next page			

Table 4.8 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_AUTH_RBAC_001	Verify Librarian can access Librarian-specific features (e.g., User Management)	• Login as Librarian. • Attempt to navigate to 'User Management' section. • Attempt to navigate to 'Book Management' (add/edit/delete books).	• Librarian successfully accesses 'User Management' page. • Librarian can view and interact with user administration tools. • Librarian successfully accesses 'Book Management' page and can perform CRUD operations on books.
TC_AUTH_RBAC_002	Verify Member cannot access Librarian-specific features (e.g., User Management)	• Login as Member. • Check if 'User Management' link/button is visible. • Attempt to navigate directly to the User Management URL (if known). • Check if 'Add Book' or 'Edit Book' options are available.	• 'User Management' link/button is not visible or is disabled. • Direct navigation to User Management URL redirects to an error page or member dashboard. • 'Add Book'/'Edit Book' options are not available. Access is denied.
TC_AUTH_RBAC_003	Verify Member can access Member-specific features (e.g., Search Books, View Borrowal History)	• Login as Member. • Attempt to navigate to 'Search Books' page. • Attempt to view 'My Borrowal History'.	• Member successfully accesses 'Search Books' page and can search. • Member successfully accesses 'My Borrowal History' and can view their own history.
Continued on next page			

Table 4.8 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_AUTH_RBAC_004	Verify guest (not logged in) user access to public pages and restriction from protected pages	• Ensure no user is logged in (clear session/cookies or use incognito window). • Attempt to navigate to the login page. • Attempt to navigate to a public book search page (if applicable). • Attempt to navigate to 'User Management' URL. • Attempt to navigate to 'My Borrowal History' URL.	• Login page is accessible. • Public book search page is accessible (if such a feature exists for guests). • Access to 'User Management' URL is denied, likely redirecting to login page. • Access to 'My Borrowal History' URL is denied, likely redirecting to login page.

4.3.1.2 Entity Management Testing

Table 4.9: Entity Management Test Cases

Test Case ID	Test Title	Test Steps	Expected Results
Sub-Section: Book Management (SRS 3.3)			
TC_BOOK_CREATE_001	Successful new book creation by Librarian	• Login as Librarian. • Navigate to 'Book Management'. • Click 'New Book'. • Fill all required fields (Name, ISBN) with valid, unique data. Select existing Author and Genre. • Submit the form.	• System accepts the data. • A success message is displayed. • The new book appears in the book list.
Continued on next page			

Table 4.9 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_BOOK_CREATE_002	Attempt to create a book with missing required fields	• Login as Librarian. • Navigate to 'Book Management'. • Click 'New Book'. • Leave the 'Name' or 'ISBN' field blank. • Submit the form.	• System rejects the creation. • An error message indicating required fields is displayed. • The book is not created.
TC_BOOK_CREATE_003	(RBAC) Verify Member cannot create a book	• Login as a Member. • Navigate to 'Book Management'.	• The 'New Book' button/option is not visible or is disabled. • Direct access to the create book API endpoint (e.g., POST /api/book/add) should be denied.
TC_BOOK_READ_001	Verify all users can view the list of books	• Login as Librarian. Navigate to 'Book Management'. • Logout. Login as Member. Navigate to 'Book Management'.	• Both Librarian and Member can successfully view the list of all books.
TC_BOOK_UPDATE_001	Successful book update by Librarian	• Login as Librarian. • Navigate to 'Book Management'. • Select a book and choose to edit it. • Modify a field (e.g., the summary). • Save the changes.	• System accepts the changes. • A success message is displayed. • The book list/details reflect the updated information.
Continued on next page			

Table 4.9 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_BOOK_DELETE_001	Successful book deletion by Librarian	• Pre-condition: Create a book that has no associated borrowals. • Login as Librarian. • Navigate to 'Book Management'. • Select the book and click 'Delete'. • Confirm the deletion.	• System accepts the deletion. • A success message is displayed. • The book is removed from the book list.
TC_BOOK_DELETE_002	(Referential Integrity) Attempt to delete a book that is currently borrowed	• Pre-condition: A book exists and is part of an active borrowing record. • Login as Librarian. • Navigate to 'Book Management'. • Attempt to delete the borrowed book.	• System prevents the deletion. • An appropriate error message is displayed (e.g., "Cannot delete a book with active borrowals"). • The book is not deleted.
Sub-Section: Author Management (SRS 3.4)			
TC_AUTHOR_CREATE_001	Successful new author creation by Librarian	• Login as Librarian. • Navigate to 'Author Management'. • Click 'New Author'. • Fill the required 'Name' field with valid data. • Submit the form.	• A success message is displayed. • The new author appears in the authors list.
Continued on next page			

Table 4.9 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_AUTHOR_DELETE_001	(Referential Integrity) Attempt to delete an author linked to a book	• Pre-condition: An author is assigned to at least one book in the system. • Login as Librarian. • Navigate to 'Author Management'. • Attempt to delete the author.	• System prevents the deletion. • An error message is displayed (e.g., "Cannot delete author assigned to existing books"). • The author is not deleted.
Sub-Section: Genre Management (SRS 3.5)			
TC_GENRE_CREATE_001	Successful new genre creation by Librarian	• Login as Librarian. • Navigate to 'Genre Management'. • Click 'New Genre'. • Fill the required 'Name' and 'Description' fields with valid data. • Submit the form.	• A success message is displayed. • The new genre appears in the genres list.
TC_GENRE_DELETE_001	(Referential Integrity) Attempt to delete a genre linked to a book	• Pre-condition: A genre is assigned to at least one book in the system. • Login as Librarian. • Navigate to 'Genre Management'. • Attempt to delete the genre.	• System prevents the deletion. • An error message is displayed (e.g., "Cannot delete genre assigned to existing books"). • The genre is not deleted.
Sub-Section: User Management (SRS 3.7)			
Continued on next page			

Table 4.9 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_USER_CREATE_001	Successful new user (Member) creation by Librarian	• Login as Librarian. • Navigate to 'User Management'. • Click 'New User'. • Fill required fields (Name, Email, Password) with valid, unique data. Set 'isAdmin' flag to false. • Submit the form.	• A success message is displayed. • The new user appears in the user list.
TC_USER_CREATE_002	Attempt to create a user with an existing email	• Login as Librarian. • Navigate to 'User Management'. • Attempt to create a new user with an email address that is already in use.	• System rejects the creation. • An error message like "User already exists" is displayed.
TC_USER_READ_001	(RBAC) Verify Member cannot access User Management	• Login as a Member.	• 'User Management' link/feature is not visible or is disabled. • Direct navigation to the user management URL is denied.
TC_USER_DELETE_001	(Referential Integrity) Attempt to delete a member with active borrowals	• Pre-condition: A member has one or more active (not returned) borrowal records. • Login as Librarian. • Navigate to 'User Management'. • Attempt to delete the member.	• System prevents the deletion. • An error message is displayed (e.g., "Cannot delete member with active borrowals"). • The user is not deleted.
Sub-Section: Borrowal Management (SRS 3.6)			
Continued on next page			

Table 4.9 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_BORROW_CREATE_001	Successful borrowal creation by Member	• Pre-condition: A book is available (isAvailable=true). • Login as Member. • Navigate to 'Borrowal Management'. • Click 'New Borrowal' and select an available book. • Submit the form.	• A new borrowal record is created with 'Borrowed' status. • A success message is displayed. • The book's availability status is updated to 'false'.
TC_BORROW_CREATE_002	Attempt to borrow an unavailable book	• Pre-condition: A book is unavailable (isAvailable=false). • Login as Member. • Attempt to create a borrowal for the unavailable book.	• The system prevents the borrowal. • An appropriate error message is displayed (e.g., "Book is not available").
TC_BORROW_READ_001	Verify Member can only see their own borrowal history	• Pre-condition: Multiple members have borrowal records. • Login as Member 1. • Navigate to 'Borrowal Management'.	• The list only displays borrowals where the memberId matches Member 1's ID. • Borrowals for other members are not visible.
Continued on next page			

Table 4.9 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_BORROW_UPDATE_001	Successful borrowal update by Librarian (mark as returned)	• Pre-condition: An active borrowal exists. • Login as Librarian. • Navigate to 'Borrowal Management'. • Select the active borrowal and edit it. • Change the status to 'Returned'. • Save the changes.	• The borrowal record status is updated. • The associated book's 'isAvailable' status should ideally be updated to 'true'. • A success message is displayed.
Sub-Section: Review Management (SRS 3.8)			
TC_REVIEW_CREATE_001	Successful review creation by Member	• Pre-condition: Based on assumed member capabilities if UI were present. • Login as Member. • Navigate to a specific book's detail page. • Submit a new review with a star rating and text.	• A success message is displayed. • The new review is visible on the book's page.
TC_REVIEW_CREATE_002	Attempt to create a review with invalid data	• Login as Member. • Attempt to submit a review with a required field missing (e.g., star rating) or an invalid value (e.g., 6 stars).	• System rejects the submission. • An appropriate error message is displayed.
Continued on next page			

Table 4.9 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_REVIEW_DELETE_001	Successful review deletion by Librarian	• Pre-condition: A review exists for a book. • Login as Librarian. • Navigate to the book or a review management section. • Select a review and delete it.	• The review is successfully deleted. • The review is no longer visible.

4.3.1.3 Use Case Testing

Table 4.10: Use Case Test Cases

Test Case ID	Test Title / Use Case	Test Steps	Expected Results
Sub-Section: UC-002: Add New Book (Librarian)			
TC_UC_BOOK_001	Successful addition of a new book (Main Flow)	• As a logged-in Librarian, navigate to the Book Management page ('/books'). • Click the "New Book" button. • Fill in all required fields (Name, ISBN) with valid data. • Select a valid Author and Genre from the provided lists. • Ensure "Is Available" is set to true. • Submit the form.	• The system creates a new book record via 'POST /api/book/add'. • A success message is displayed (e.g., "Book added"). • The "Add Book" form closes. • The new book appears in the list on the Book Management page.
Continued on next page			

Table 4.10 – continued from previous page

Test Case ID	Test Title / Use Case	Test Steps	Expected Results
TC_UC_BOOK_002	Attempt to add a book with missing required fields (Alternative Flow A1)	- As a logged-in Librarian, navigate to the "Add Book" form. - Fill in the form but leave a required field (e.g., Name) blank. - Submit the form.	- The system rejects the submission due to a validation error. - An error message is displayed, indicating the required field is missing. - The form remains open for correction. - The book is not added to the database.
TC_UC_BOOK_003	Attempt to add a book that fails backend validation (Alternative Flow A2)	- As a logged-in Librarian, navigate to the "Add Book" form. - Fill in all fields with data that is syntactically valid on the client-side but might trigger a server error (e.g., a duplicate ISBN if the server enforces uniqueness). - Submit the form.	- The system displays a generic error message from the API (e.g., "Something went wrong, please try again"). - The book is not added to the database.
Sub-Section: UC-003: Borrow a Book (Member)			
TC_UC_BORROW_001	Successful borrowing of an available book (Main Flow)	- As a logged-in Member, navigate to the Borrowal Management page ('/borrowals'). - Click the "New Borrowal" button. - Select a book that is marked as available from the list. - Verify that the Member field is correctly auto-populated. - Submit the form.	- A new borrowal record is created with "Borrowed" status via 'POST /api/borrowal/add'. - The availability of the book is updated to 'false'. - A success message is displayed. - The member's borrowal list is updated to show the new borrowal.
Continued on next page			

Table 4.10 – continued from previous page

Test Case ID	Test Title / Use Case	Test Steps	Expected Results
TC_UC_BORROW_002	Attempt to borrow an unavailable book (Alternative Flow A1)	- As a logged-in Member, navigate to the "Add Borrowal" form. - Attempt to select a book that is not available (isAvailable=false).	- The UI should ideally prevent the selection of an unavailable book. - If selection is possible, the system should prevent submission or display an error message (e.g., "Book is not available") upon submission attempt. - No borrowal record is created.
TC_UC_BORROW_003	Verify correct data population in borrowal form	- As a logged-in Member, open the "Add Borrowal" form. - Observe the date fields.	- The Member field is correctly auto-populated with the current user's ID. - The Borrowed Date is auto-populated to the current date. - The Due Date is auto-populated to a future date (e.g., 14 days from current). - The initial status is correctly set (e.g., "Borrowed").
Sub-Section: UC-005: View Borrowal History (Member)			
Continued on next page			

Table 4.10 – continued from previous page

Test Case ID	Test Title / Use Case	Test Steps	Expected Results
TC_UC_HISTORY_001	View non-empty borrowal history (Main Flow)	- Log in as a Member who has previously borrowed one or more books. - Navigate to the Borrowal Management page ('/borrowals').	- The system fetches all borrowals and the client filters them. - A list/table is displayed showing *only* the records where the 'memberId' matches the logged-in Member's ID. - The displayed data (Book Name, Borrowed Date, Due Date, Status) is accurate for each record.
TC_UC_HISTORY_002	View empty borrowal history (Alternative Flow A1)	- Log in as a new Member with no borrowal records. - Navigate to the Borrowal Management page ('/borrowals').	The system displays a message indicating there is no borrowal history (e.g., "No borrowals found").
TC_UC_HISTORY_003	Verify role-based view of borrowal history	- Prerequisite: There are borrowals from multiple members in the system. - Log in as a Librarian and navigate to the Borrowal Management page ('/borrowals'). - Log out and log in as a Member (who has at least one borrowal) and navigate to the same page.	- As the Librarian, all borrowal records for all members are visible. - As the Member, only their own borrowal history is visible.
Note: UC-001 (User Login) and UC-004 (Register New User) are detailed in the Authentication & Authorization test plan.			

4.3.1.4 State Transition Testing

Table 4.11: State Transition Test Cases

Test Case ID	Test Title	Test Steps	Expected Results
Entity: Borrowal Record (Based on 'status' attribute)			
TC_STATE_BORROW_001	Valid Transition: (None) to 'Borrowed'	• Login as a Member. • Navigate to the Borrowal Management page. • Create a new borrowal record for an available book. • Submit the form.	• A new borrowal record is successfully created. • The status of the new record is set to "Borrowed". • A success message is displayed.
TC_STATE_BORROW_002	Valid Transition: 'Borrowed' to 'Returned'	• Precondition: A book is in 'Borrowed' status by a member. • Login as Librarian. • Navigate to Borrowal Management. • Locate the 'Borrowed' record. • Update the record's status to 'Returned' before its due date.	• The system accepts the update. • The borrowal record's status changes from "Borrowed" to "Returned". • A success message is displayed.
TC_STATE_BORROW_003	Valid Transition: 'Borrowed' to 'Overdue'	• Precondition: A book is in 'Borrowed' status by a member. • Manually adjust system time or the record's due date to be in the past. • As a Librarian, view the borrowal record. (Or trigger a system check for overdue items).	• The borrowal record's status automatically (or upon view) changes from "Borrowed" to "Overdue". • The status is displayed correctly in the UI.
TC_STATE_BORROW_004	Valid Transition: 'Overdue' to 'Returned'	• Precondition: A book is in 'Overdue' status. • Login as Librarian. • Locate the 'Overdue' borrowal record. • Update the record's status to 'Returned'.	• The system accepts the update. • The borrowal record's status changes from "Overdue" to "Returned". • A success message is displayed.
			Continued on next page

Table 4.11 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_STATE_BORROW_005	Invalid Transition: 'Returned' to 'Borrowed'	• Precondition: A borrowal record has a 'Returned' status. • As a Librarian, attempt to edit the same record and change its status back to 'Borrowed'.	• The system should reject the change. • The UI should not provide an option for this transition, or it should result in an error if attempted via API. • An appropriate error message is displayed (e.g., "Cannot revert a returned item").
Entity: Book (Based on 'isAvailable' attribute)			
TC_STATE_BOOK_001	Valid Transition: 'Available' to 'Unavailable'	• Precondition: A book exists with 'isAvailable' status set to true. • As a Member, create a new borrowal for that specific book. • After the borrowal is confirmed, view the details of that book.	• The borrowal transaction is successful. • The 'isAvailable' status of the book is updated to false. • The book's listing correctly shows it as unavailable.
TC_STATE_BOOK_002	Valid Transition: 'Unavailable' to 'Available'	• Precondition: A book has 'isAvailable' status set to false due to being borrowed. • As a Librarian, locate the corresponding borrowal record. • Update the borrowal record status to 'Returned'. • View the details of the book again.	• The 'isAvailable' status of the book is updated to true. • The book's listing correctly shows it as available for borrowing.
Continued on next page			

Table 4.11 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_STATE_BOOK_003	Invalid Transition: Attempt to borrow an 'Unavailable' book	• Precondition: A book has 'isAvailable' status set to false. • As a Member, attempt to create a new borrowing for that specific book.	• The system prevents the borrowing request from being submitted. • An appropriate error message is displayed (e.g., "Book is currently unavailable"). • No new borrowing record is created.
Entity: User Session (Based on login/logout actions)			
TC_STATE_SESSION_001	Valid Transition: 'Logged-Out' to 'Logged-In'	• Start from a logged-out state (e.g., in an incognito window). • Navigate to the login page. • Enter valid credentials for a 'Member'. • Click 'Login'.	• User session is established. • User is redirected to the member landing page (e.g., /books). • User can access member-specific pages.
TC_STATE_SESSION_002	Valid Transition: 'Logged-In' to 'Logged-Out'	• Precondition: A user is logged in. • Click the 'Logout' button/link.	• The user's session is terminated. • User is redirected to the login page.
TC_STATE_SESSION_003	Invalid Transition: Accessing protected page when 'Logged-Out'	• Start from a logged-out state. • Attempt to directly navigate to a protected URL (e.g., /borrowals, /users).	• Access to the protected page is denied. • User is redirected to the login page.
TC_STATE_SESSION_004	State Persistence: Verify session remains 'Logged-In' after page refresh	• Login as any user. • Navigate to any page other than the login page. • Refresh the web browser page.	• The user remains logged in. • The user stays on the same page. • No redirection to the login page occurs.

4.3.1.5 Specific Feature Testing

Table 4.12: Specific Feature Test Cases

Test Case ID	Test Title	Test Steps	Expected Results
Sub-Section: Dashboard (Librarian)			
TC_DASH_001	Verify Librarian can access the Dashboard	• Login as Librarian. • Navigate to the Dashboard.	• User is successfully redirected to the Librarian dashboard at '/dashboard'. • Dashboard displays overview statistics and charts.
TC_DASH_002	Verify Member cannot access the Dashboard	• Login as a 'Member' user. • Attempt to navigate directly to the '/dashboard' URL.	• Access is denied. • User is redirected to the member landing page (e.g., '/books') or an error page.
Sub-Section: Book Management (CRUD)			
TC_BOOK_ADD_001	Successful new book creation by Librarian	• Login as Librarian. • Navigate to 'Book Management' (/books). • Click "New Book" button. • Fill in all required fields (Name, ISBN) with valid data. • Select existing Author and Genre. • Submit the form.	• A success message is displayed. • The new book record is created and appears in the book list.
TC_BOOK_ADD_002	Attempt to create a book with missing required fields	• Login as Librarian. • Navigate to 'Book Management'. • Open the "Add Book" form. • Leave a required field (e.g., Name) blank. • Submit the form.	• The system rejects the submission. • An error message indicating the required field is missing is displayed. • The book is not added to the database.
TC_BOOK_VIEW_001	Verify Member can view list of books and book details	• Login as Member. • Navigate to 'Book Management' (/books). • Click on a specific book in the list.	• Member can successfully view the list of all books. • Member can successfully view the details of a specific book.
Continued on next page			

Table 4.12 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_BOOK_UPD_001	Successful update of a book's details by Librarian	• Login as Librarian. • Navigate to 'Book Management'. • Select a book to edit. • Change a field (e.g., update the summary). • Save the changes.	• A success message is displayed. • The updated information is reflected correctly in the book's detail view.
TC_BOOK_DEL_001	Successful deletion of a book by Librarian	• Login as Librarian. • Navigate to 'Book Management'. • Select a book to delete. • Confirm the deletion in the dialog.	• A success message is displayed. • The book is removed from the book list.
TC_BOOK_ACCESS_001	Verify Member cannot access book CRUD operations	• Login as Member. • Navigate to 'Book Management'.	• "Add Book", "Edit Book", and "Delete Book" UI elements are not visible or are disabled. • Direct access to corresponding API endpoints is denied.
Sub-Section: Borrowal Management			
TC_BORW_ADD_001	Successful borrowal request by Member for an available book	• Login as Member. • Navigate to 'Borrowal Management' (/borrowals). • Click "New Borrowal". • Select a book that is marked as available. • Submit the form.	• A success message is displayed. • A new borrowal record is created with 'Borrowed' status and correct dates. • The new record appears in the member's borrowal history.
TC_BORW_ADD_002	Attempt to borrow a book that is not available	• Login as Member. • Navigate to 'Borrowal Management'. • Attempt to create a new borrowal for a book that is not available.	• The UI should clearly indicate the book is not available. • The system prevents the submission or displays a clear error message.
Continued on next page			

Table 4.12 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_BORW_VIEW_001	Verify Member can only view their own borrowal history	• Login as a Member who has borrowal records. • Navigate to 'Borrowal Management' (/borrowals).	• The list displays only the borrowals associated with the logged-in member's ID. • Records of other members are not visible.
TC_BORW_VIEW_002	Verify Librarian can view all borrowal records	• Login as Librarian. • Navigate to 'Borrowal Management'.	• A list of all borrowal records for all members is displayed.
TC_BORW_UPD_001	Successful update of borrowal status by Librarian	• Login as Librarian. • Navigate to 'Borrowal Management'. • Select an existing 'Borrowed' record. • Update its status to 'Returned'. • Save the change.	• A success message is displayed. • The record's status is correctly updated in the list.
Sub-Section: User Management (by Librarian)			
TC_USER_VIEW_001	Verify Librarian can view list of all users	• Login as Librarian. • Navigate to 'User Management' (/users).	• A list of all users (both Librarians and Members) is displayed correctly.
TC_USER_UPD_001	Successful update of a user's details by Librarian	• Login as Librarian. • Navigate to 'User Management'. • Select a user to edit. • Update a non-critical field (e.g., Phone number). • Save the changes.	• A success message is displayed. • The user's details are correctly updated.
TC_USER_DEL_001	Successful deletion of a user by Librarian	• Login as Librarian. • Navigate to 'User Management'. • Select a user account to delete. • Confirm the deletion.	• A success message is displayed. • The user is removed from the list of users. • The deleted user can no longer log in.
Sub-Section: Review Management (Assumed Admin UI)			
Continued on next page			

Table 4.12 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_REV_ADD_001	Successful addition of a review by Librarian (Admin action)	• Login as Librarian. • Navigate to a book's detail page. • Use an administrative function to add a review for the book. • Enter a star rating and review text. • Submit the review.	• A success message is displayed. • The new review appears in the list of reviews for that book.
TC_REV_VIEW_001	Verify any user can view reviews for a book	• As any logged-in user (or guest, if public). • Navigate to the detail page of a book that has reviews.	• The list of reviews for the book is displayed correctly.
TC_REV_DEL_001	Successful deletion of a review by Librarian	• Login as Librarian. • Navigate to a book with reviews. • Select a specific review to delete. • Confirm the deletion.	• A success message is displayed. • The review is removed from the book's list of reviews.

4.3.2 API Testing

API testing is performed to ensure the reliability, functionality, performance, and security of the application's API layer.

Table 4.13: API Test Cases

Test Case ID	Objective	API Endpoint & Method	Request Body / Parameters & Expected Results
Sub-Section: Authentication API ('/api/auth')			
TC_API_AUTH_001	Successful Librarian Login	POST '/api/auth/login'	Request Body (JSON): <pre> 1 { 2 "email": " librarian@example.com", 3 "password": " valid_password" 4 } </pre> Expected Result: • Status Code: 200 OK • Response Body: Contains user object with 'isAdmin: true' and an authentication token/session cookie.
TC_API_AUTH_002	Successful Member Login	POST '/api/auth/login'	Request Body (JSON): <pre> 1 { 2 "email": "member@example. com", 3 "password": " valid_password" 4 } </pre> Expected Result: • Status Code: 200 OK • Response Body: Contains user object with 'isAdmin: false' and an authentication token/session cookie.
TC_API_AUTH_003	Login with invalid password	POST '/api/auth/login'	Request Body (JSON): <pre> 1 { 2 "email": " librarian@example.com", 3 "password": " invalid_password" 4 } </pre> Expected Result: • Status Code: 401 Unauthorized (or similar) • Response Body: Contains an appropriate error message.

Continued on next page

Table 4.13 – continued from previous page

Test Case ID	Objective	API Endpoint & Method	Request Body / Parameters & Expected Results
TC_API_AUTH_004	Librarian registers a new member	POST ‘/api/auth/register’ (Requires Librarian auth)	Request Body (JSON): <pre> 1 { 2 "name": "New Member", 3 "email": "new. 4 member@example.com", 5 "password": " 6 strong_password", 7 "isAdmin": false 8 } </pre> Expected Result: • Status Code: 201 Created • Response Body: Confirmation message. The user record is created in the database with a hashed password.
TC_API_AUTH_005	Attempt to register with an existing email	POST ‘/api/auth/register’ (Requires Librarian auth)	Request Body (JSON): <pre> 1 { 2 "name": "Another Member", 3 "email": "member@example. 4 com", 5 "password": " 6 another_password", 7 "isAdmin": false 8 } </pre> Expected Result: • Status Code: 400 Bad Request (or similar) • Response Body: Error message indicating the user already exists.
TC_API_AUTH_006	Member attempts to access user registration endpoint	POST ‘/api/auth/register’ (Requires Member auth)	Request Body (JSON): (any valid user data) Expected Result: • Status Code: 403 Forbidden • Response Body: Error message indicating insufficient permissions, as only librarians can register users.
TC_API_AUTH_007	Successful Logout	GET ‘/api/auth/logout’ (Requires any user auth)	Request Body: N/A Expected Result: • Status Code: 200 OK (or redirect) • The user’s session is terminated. Subsequent requests to protected endpoints should fail.
Sub-Section: Book Management API (‘/api/book’)			
Continued on next page			

Table 4.13 – continued from previous page

Test Case ID	Objective	API Endpoint & Method	Request Body / Parameters & Expected Results
TC_API_BOOK_001	Librarian adds a new book	POST ‘/api/book/add’ (Requires Librarian auth)	Request Body (JSON): <pre> 1 { 2 "name": "The Art of 3 Software Testing", 4 "isbn": "978-1118031534", 5 "isAvailable": true 6 } </pre> Expected Result: • Status Code: 201 Created • Response Body: The newly created book object.
TC_API_BOOK_002	Member attempts to add a new book	POST ‘/api/book/add’ (Requires Member auth)	Request Body (JSON): (any valid book data) Expected Result: • Status Code: 403 Forbidden • Response Body: Error message indicating insufficient permissions. Librarians have exclusive add rights.
TC_API_BOOK_003	Any user gets a list of all books	GET ‘/api/book/getAll’ (Public or Member auth)	Request Body: N/A Expected Result: • Status Code: 200 OK • Response Body: An array of book objects.
TC_API_BOOK_004	Librarian deletes a book	DELETE ‘/api/book/delete/id’ (Requires Librarian auth)	Parameters: URL parameter ‘id’ of an existing book. Expected Result: • Status Code: 200 OK (or 204 No Content) • Response Body: Success message. The book is removed from the database.
TC_API_BOOK_005	Unauthenticated user attempts to delete a book	DELETE ‘/api/book/delete/id’ (No auth)	Parameters: URL parameter ‘id’ of an existing book. Expected Result: • Status Code: 401 Unauthorized • Response Body: Error message indicating authentication is required.
Sub-Section: Borrowal Management API (‘/api/borrowal’)			

Continued on next page

Table 4.13 – continued from previous page

Test Case ID	Objective	API Endpoint & Method	Request Body / Parameters & Expected Results
TC_API_BORROW_1	Member borrows an available book for themselves	POST ‘/api/borrowal/add’ (Requires Member auth)	Request Body (JSON): <pre> 1 { 2 "bookId": "valid_book_id", 3 "memberId": " 4 current_user_id" 5 } </pre> Expected Result: • Status Code: 201 Created • Response Body: The new borrowal record. The book’s availability may be updated.
TC_API_BORROW_2	Member attempts to borrow a book for another member	POST ‘/api/borrowal/add’ (Requires Member auth)	Request Body (JSON): <pre> 1 { 2 "bookId": "valid_book_id", 3 "memberId": " 4 another_user_id" 5 } </pre> Expected Result: • Status Code: 403 Forbidden • Response Body: Error message. The API must enforce that the ‘memberId’ in the request matches the authenticated user’s ID.
TC_API_BORROW_3	Librarian updates a borrowal status (e.g., to ‘Returned’)	PUT ‘/api/borrowal/update/id’ (Requires Librarian auth)	Parameters: URL parameter ‘id’ of an existing borrowal. Request Body (JSON): <pre> 1 { 2 "status": "Returned" 3 } </pre> Expected Result: • Status Code: 200 OK • Response Body: The updated borrowal object.
TC_API_BORROW_4	Member attempts to view all borrowals in the system	GET ‘/api/borrowal/getAll’ (Requires Member auth)	Request Body: N/A Expected Result: • Status Code: 200 OK • Response Body: An array containing only the borrowal records for the authenticated member. The API should not leak other users’ data.
Sub-Section: Security & Input Validation			
Continued on next page			

Table 4.13 – continued from previous page

Test Case ID	Objective	API Endpoint & Method	Request Body / Parameters & Expected Results
TC_API_SEC_001	Verify password hash is not returned on user lookup	GET ‘/api/user/get/id’ (Requires Librarian auth)	Parameters: URL parameter ‘id’ for any user. Expected Result: • Status Code: 200 OK • Response Body: The user object must not contain the ‘hash’ or ‘salt’ fields.
TC_API_SEC_002	Attempt basic SQL Injection (or NoSQL equivalent)	POST ‘/api/auth/login’	Request Body (JSON): <pre> 1 { 2 "email": " ' OR '1'='1", 3 "password": " ' OR '1'='1" 4 } </pre> Expected Result: • Status Code: 401 Unauthorized (or 400 Bad Request) • Response Body: Normal login failure message. The system must not be compromised. This validates server-side input validation.

4.3.3 White-Box Testing

White-box testing examines the internal logic and structure of the code. Code coverage is a key metric used to evaluate the thoroughness of our white-box testing efforts.

- **Objective:** To ensure that internal paths in the code are exercised and that the internal logic is sound.
- **Methodology:** We use Jest’s built-in coverage reporting tools, which are based on Istanbul. This helps us identify untested parts of the codebase.

4.3.3.1 Test Strategy and Coverage Criteria

4.3.3.1.1 Coverage Types Implemented

4.3.3.1.2 Statement Coverage

- **Target:** 90% statement coverage
- **Measurement:** Every executable statement tested at least once
- **Implementation:** Jest coverage reports track statement execution

4.3.3.1.3 Branch Coverage

- **Target:** 85% branch coverage
- **Measurement:** All conditional branches (if/else, switch cases) tested
- **Implementation:** Test cases cover both true and false conditions

4.3.3.1.4 Condition Coverage

- **Target:** 80% condition coverage
- **Measurement:** Individual conditions in complex boolean expressions tested
- **Implementation:** Separate test cases for each condition combination

4.3.3.1.5 Path Coverage

- **Target:** 75% path coverage for critical functions
- **Measurement:** Independent execution paths through functions tested
- **Implementation:** Test cases designed to exercise different execution paths

4.3.3.2 Complexity Analysis

4.3.3.2.1 Cyclomatic Complexity Assessment

Critical functions analyzed for complexity:

- **authController.registerUser:** Complexity = 8 (High)
- **authController.loginUser:** Complexity = 7 (Medium-High)
- **bookController.addBook:** Complexity = 9 (High)
- **User.setPassword:** Complexity = 2 (Low)
- **User.isValidPassword:** Complexity = 2 (Low)

4.3.3.3 Server-Side White-Box Testing

4.3.3.3.1 Model Testing

4.3.3.3.1.1 User Model (TC_WB_USER_001 - TC_WB_USER_017)

File: server/__tests__/unit/models/user.test.js

Coverage Areas:

- Schema validation for required fields (name, email, isAdmin, photoUrl)
- Optional field validation (dob, phone)
- Password hashing logic using crypto.pbkdf2Sync
- Password validation logic with salt comparison
- Error handling for crypto operations
- Integration testing of complete password flow

Key Test Cases:

Test ID	Description	Coverage Type
TC_WB_USER_001	Should require name field	Statement
TC_WB_USER_006	Should generate random salt using crypto.randomBytes	Branch

TC_WB_USER_007	Should hash password with correct parameters	Path
TC_WB_USER_011	Should return true when hashes match	Condition
TC_WB_USER_015	Complete password set and validation flow	Integration
TC_WB_USER_016	Should handle crypto errors gracefully	Error Path

4.3.3.3.1.2 Book Model

(TC_WB_BOOK_MODEL_001 - TC_WB_BOOK_MODEL_026)

File: server/__tests__/unit/models/book.test.js

Coverage Areas:

- Schema validation for required fields (name, isbn, isAvailable)
- Optional field validation (authorId, genreId, summary, photoUrl)
- Data type validation (String, Boolean, ObjectId)
- Edge cases (empty strings, null values, unicode characters)
- Mongoose integration validation

4.3.3.3.2 Controller Testing

4.3.3.3.2.1 Authentication Controller

(TC_WB_AUTH_001 - TC_WB_AUTH_017)

File: server/__tests__/unit/controllers/authController.test.js

Coverage Areas:

- Input validation logic flow (email → password → name → password strength)
- Database interaction error handling
- User existence checking logic
- Password validation integration
- Passport authentication flow
- Session management error handling

Validation Flow Analysis:

```

1 // White-box testing reveals exact validation order:
2 1. if (!req.body.email) -> return emailRequired error
3 2. if (!req.body.password) -> return passwordRequired error
4 3. if (!req.body.name) -> return nameRequired error
5 4. if (req.body.password.length < 8) -> return weakPassword error
6 5. User.findOne() -> check existing user
7 6. Create new user if validation passes

```

Listing 4.11: Authentication Validation Flow

4.3.3.3.2 Book Controller

(TC_WB_BOOK_001 - TC_WB_BOOK_008)

File: server/__tests__/unit/controllers/bookController.test.js

Coverage Areas:

- ObjectId validation using mongoose.Types.ObjectId.isValid()
- Input validation sequence (title → authorId → genreId → isbn)
- ISBN duplication checking logic
- Database aggregation pipeline testing
- Error handling for database operations

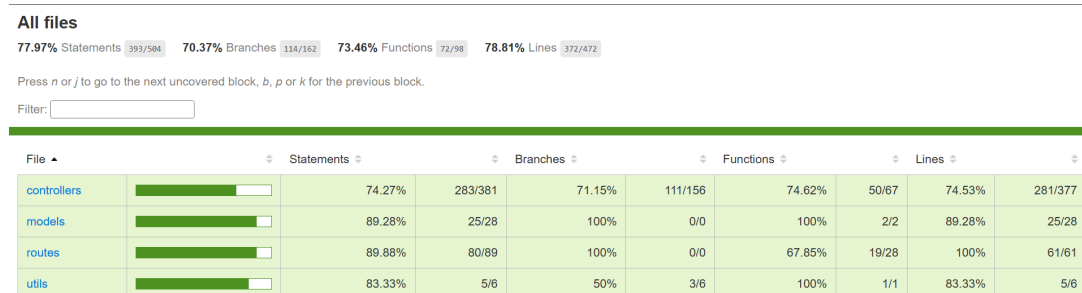


Figure 4.2: Jest Code Coverage Report

4.3.4 Non-Functional Testing

Non-functional testing evaluates aspects of the system that are not related to specific functions but are critical to user satisfaction and system integrity.

4.3.4.1 Performance Testing

Table 4.15: Performance Test Cases

Test Case ID	Test Title	Test Steps	Expected Results
Sub-Section: UI Load & Responsiveness Testing			
Continued on next page			

Table 4.15 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_PERF_UI_001	Page load time for book list with few records	• Precondition: Database contains a small number of records (e.g., < 50 books). • Login as a Librarian or Member. • Navigate to the book list page ('/books'). • Measure the time from navigation start to when the page is fully rendered and interactive.	• The page load time should be within the acceptable baseline (e.g., < 2 seconds). • The UI is responsive and usable immediately after rendering.
TC_PERF_UI_002	Page load time for book list with many records (Volume)	• Precondition: Database contains a large number of records (e.g., > 5,000 books). • Login as a Librarian or Member. • Navigate to the book list page ('/books'). • Measure the time until the page is fully rendered and interactive.	• The page load time meets the defined threshold for large data sets (e.g., < 5 seconds). • UI remains responsive; features like sorting or filtering might show a loading indicator but do not freeze the application.
TC_PERF_UI_003	UI responsiveness during data entry on a high-load page	• Navigate to a data-heavy page (e.g., Librarian Dashboard with many widgets). • While the page is rendering or widgets are loading data, attempt to interact with a simple UI element (e.g., open a dropdown menu).	• User interactions are not blocked. • The UI responds to input within a reasonable time (e.g., < 500ms) without noticeable lag.
Sub-Section: API Response Time Testing			
Continued on next page			

Table 4.15 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_PERF_API_001	API response time for fetching all books (Normal Load)	• Precondition: Database contains a moderate number of book records (e.g., 500). • Using an API testing tool (e.g., Postman), send a GET request to the ‘/api/book/getAll‘ endpoint. • Measure the server response time.	• The API response time is within the defined benchmark (e.g., < 300ms). • The HTTP status code is 200 OK.
TC_PERF_API_002	API response time for creating a new entity (e.g., Borrowal)	• Using an API testing tool, send a POST request with a valid payload to the ‘/api/borrowal/add‘ endpoint. • Measure the server response time.	• The API response time for the creation operation is within the benchmark (e.g., < 400ms). • The HTTP status code is 201 Created (or similar success code).
TC_PERF_API_003	API response time for user authentication	• Using an API testing tool, send a POST request with valid user credentials to the ‘/api/auth/login‘ endpoint. • Measure the server response time.	• Authentication is a critical path; response time should be fast (e.g., < 250ms). • The HTTP status code is 200 OK.
Sub-Section: Volume Testing			
Continued on next page			

Table 4.15 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_PERF_VOL_001	System performance with large data volume in multiple tables	• Precondition: Populate the database with an abnormally large volume of data: – 20,000+ Books – 5,000+ Users – 50,000+ Borrowal records • Perform a series of common user actions (e.g., login, search for a book, view user list, view borrowal history).	• The system remains stable and does not crash or become unresponsive. • Data retrieval and display operations complete within defined acceptable limits, though slower than with few records. • Database queries do not time out.
Sub-Section: Stress Testing			
TC_PERF_STRS_001	System stability under high concurrent user load	• Precondition: Use a load testing tool (e.g., JMeter, Gatling). • Simulate a high number of concurrent users (e.g., 100 users) performing actions simultaneously for a sustained period (e.g., 15 minutes). Actions include: – Logging in – Fetching book lists – Adding new borrowals	• The system remains operational and does not crash. • The average API response time degradation is within an acceptable percentage (e.g., does not increase by more than 50 • The error rate (e.g., HTTP 5xx errors) is below a minimal threshold (e.g., < 1
Continued on next page			

Table 4.15 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_PERF_STRS_002	System recovery after a stress period	- Following the completion of the TC_PERF_STRS_001 test, stop the load generation. - Wait for a brief period (e.g., 2 minutes). - Perform a single-user check by navigating key pages (Login, Books, Dashboard).	- The system recovers and returns to normal performance levels. - Response times for the single user should return to the baseline measurements recorded in the API and UI tests. - No data corruption has occurred.

4.3.4.2 Security Testing

Table 4.16: Security Test Cases

Test Case ID	Test Title	Test Steps	Expected Results
Sub-Section: Access Control & Authorization (RBAC)			
TC_SEC_RBAC_001	Verify Member cannot access Librarian-only URLs	- Login as a 'Member' user. - Attempt to navigate directly to a Librarian-only URL (e.g., '/users', '/dashboard').	- Access is denied. - The system redirects the user to an appropriate page (e.g., login page, member dashboard) and does not display the requested admin page.
TC_SEC_RBAC_002	Verify unauthenticated user cannot access any protected pages	- Ensure no user is logged in (e.g., use a new browser session or clear cookies). - Attempt to navigate directly to any protected URL (e.g., '/books', '/users', '/borrowals').	- Access is denied for all protected URLs. - The system redirects the user to the login page.
TC_SEC_RBAC_003	Member attempts to perform a Librarian action via API	- Login as a 'Member' user and obtain the session token. - Use an API tool (e.g., Postman) to send a request to a Librarian-only API endpoint (e.g., 'DELETE /api/user/delete/:id' with a valid user ID).	- The API returns an authorization error (e.g., 403 Forbidden or 401 Unauthorized). - The action (e.g., deleting a user) is not performed. The data remains unchanged in the database.
Sub-Section: Input Validation & Injection Attacks			
Continued on next page			

Table 4.16 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_SEC_INJ_001	Attempt basic Cross-Site Scripting (XSS) in UI form fields	- Login as Librarian. - Navigate to the 'Add Book' form. - In a text field (e.g., 'Summary'), enter a basic XSS payload like <code><script>alert('XSS')</script></code>. - Submit the form to create the book. - View the details of the newly created book.	- The script is not executed. No alert box appears. - The input is properly sanitized and displayed as plain text (e.g., <code>&lt;script&gt;alert('XSS')&lt;/script&gt;</code>) or the input is rejected.
TC_SEC_INJ_002	Attempt basic NoSQL injection in login form	- Navigate to the login page. - In the email/username field, enter a NoSQL injection payload (e.g., <code>{ \$ne: null }</code>). - Enter any password and submit.	- The login attempt fails. - The system returns a generic error message like <code>"Invalid username or password"</code> and does not crash or reveal system information. - The application is not bypassed.
TC_SEC_INJ_003	Attempt insecure direct object reference (IDOR) via API	- Login as 'Member A'. - Navigate to 'My Borrowal History' and note the URL or API call used to fetch the data (e.g., 'GET /api/borrowal/getAll' which is then filtered by the client). - Using an API tool, attempt to request details of a specific borrowal that belongs to 'Member B' by guessing or obtaining its ID (e.g., 'GET /api/borrowal/get/:borrowal_id_of_member_B').	- The API returns an authorization error (e.g., 403 Forbidden or 404 Not Found). - The details of 'Member B's borrowal record are not displayed.
Sub-Section: Session Management			
TC_SEC_SESS_001	Verify session invalidation on logout	- Login as any user. - Copy the URL of a protected page (e.g., '/books'). - Click the 'Logout' button. The user is redirected to the login page. - Click the browser's 'Back' button or paste the copied URL into the address bar.	- The protected page is not displayed. - The user remains on the login page or is redirected back to it, confirming the session was terminated.

Continued on next page

Table 4.16 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_SEC_SESS_002	Verify secure session cookie attributes	- Login to the application. - Use browser developer tools to inspect the cookies set by the application.	- Session cookies should have the 'HttpOnly' attribute to prevent access via JavaScript. - Session cookies should have the 'Secure' attribute (if using HTTPS) to ensure they are sent only over secure connections.
Sub-Section: Data Security & Transport Layer			
TC_SEC_DATA_001	Verify sensitive data is not returned in API responses	- Login as Librarian. - Use an API tool to call an endpoint that returns user data (e.g., 'GET /api/user/get/:id'). - Inspect the JSON response body.	- The response does not contain sensitive data like the user's password hash or salt. - Only necessary and non-sensitive user data is returned (e.g., name, email, role).
TC_SEC_DATA_002	Verify secure data transmission (HTTPS)	- Access the application in a web browser. - Check the URL in the address bar. - Use browser developer tools to monitor network traffic during login and other operations.	- The connection uses HTTPS, indicated by 'https://' and a lock icon. - All data, especially login credentials, is sent over the encrypted HTTPS connection.
Sub-Section: Static Testing (Code Review)			
TC_SEC_STATIC_001	Code review for security vulnerabilities	(Led by Coders) - Systematically review backend source code, focusing on authentication, session management, and data access logic. - Use a checklist to look for common vulnerabilities (e.g., improper input validation, hardcoded secrets, use of insecure libraries, potential for injection). - Review frontend code for potential XSS vulnerabilities and improper handling of sensitive data.	- A report is generated documenting all potential security flaws found in the code. - Vulnerabilities are categorized by severity. - Recommendations for remediation are provided.

4.3.4.3 Usability Testing

Table 4.17: Usability Test Cases

Test Case ID	Objective	Test Steps	Expected Results	Status
TC-USAB-001	Verify the clarity and intuitiveness of the login page.	1. Navigate to the login page. 2. Observe the layout and labels. 3. Attempt to log in with incorrect credentials.	1. The page is uncluttered with clear input fields for "Email" and "Password". 2. A clear, non-technical error message is displayed upon failed login. 3. The purpose of the page is immediately understandable.	To Be Executed
TC-USAB-002	Assess the consistency of the main navigation menu across different pages.	1. Log in as a Librarian. 2. Navigate to Dashboard, Users, Books, and Authors pages. 3. Observe the position, style, and icons of the navigation sidebar.	The navigation sidebar remains in a consistent location and maintains the same look and feel across all pages. The active page is clearly highlighted.	To Be Executed
TC-USAB-003	Evaluate the effectiveness of user feedback for common actions.	1. Log in as a Librarian. 2. Create a new Author. 3. Update the details of an existing Book. 4. Delete a Genre.	1. A clear success message (e.g., a toast or snackbar) appears after each successful action. 2. A confirmation dialog appears before the deletion, preventing accidental data loss. 3. Feedback is timely and easy to understand.	To Be Executed
TC-USAB-004	Evaluate the clarity and usefulness of the Librarian's dashboard.	1. Log in as a Librarian. 2. View the dashboard. 3. Analyze the widgets for "Total Books", "Total Members", etc. 4. Review the charts.	1. The dashboard provides a clear, at-a-glance summary of key library statistics. 2. Charts are well-labeled and easy to interpret. 3. The visual hierarchy directs attention to important information. See Figure 4.3.	To Be Executed

Continued on next page

Table 4.17 – continued from previous page

Test Case ID	Objective	Test Steps	Expected Results	Status
TC-USAB-005	Test the ease of creating a new user.	1. Navigate to the "User" page. 2. Click the "New User" button. 3. Fill out the user creation form with valid data. 4. Submit the form.	The process is intuitive. Form fields are clearly labeled, and input requirements (e.g., password complexity) are clear. The new user appears in the user list immediately after creation.	To Be Executed
TC-USAB-006	Test the intuitiveness of the book management workflow.	1. Navigate to the "Book" page. 2. Use the filter/search bar to find a specific book. 3. Click to edit a book. 4. Change the book's genre and save.	The search functionality is easy to find and use. Editing a book is straightforward, with a clear form and save button. The UI updates instantly to reflect the change.	To Be Executed
TC-USAB-007	Evaluate the process of borrowing a book for a Member.	1. Log in as a Member. 2. Find an available book. 3. Click the "Borrow" button. 4. Confirm the action.	The "Borrow" action is clearly available for books that are not already on loan. The process requires minimal steps. The user receives clear confirmation that the book has been successfully borrowed.	To Be Executed
TC-USAB-008	Assess the clarity of the member's borrowing history page.	1. Log in as a Member. 2. Navigate to the "Borrowals" page. 3. View the list of current and past borrowals.	The page clearly displays essential information: book title, borrow date, due date, and status (e.g., "Borrowed", "Returned"). The layout is easy to read and understand. See Figure 4.4.	To Be Executed
TC-USAB-009	Test the ease of leaving a review for a book.	1. Log in as a Member. 2. Navigate to a book they have borrowed. 3. Find the option to leave a review. 4. Submit a rating and a comment.	The review submission form is easy to find and use. The user can easily select a rating (e.g., via stars) and type a comment. The submitted review appears promptly.	To Be Executed



Figure 4.3: Librarian Dashboard UI for Usability Evaluation

LIBRARY management

Borrowals

+ New Borrowal

Member Name	Book Name	Borrowed On	Due On	Status	
Test Member	Foundation	Invalid Date	Invalid Date	Borrowed	⋮
Test Member	1984	Invalid Date	Invalid Date	Borrowed	⋮
Test Member	To Kill a Mockingbird	Invalid Date	Invalid Date	Borrowed	⋮

Rows per page: 5 1-3 of 3 < >

Figure 4.4: Member Borrowal History UI for Usability Evaluation

For a more detailed breakdown of usability heuristics, we utilized a checklist during evaluation:

4.3.4.4 Browser Compatibility Testing

Table 4.18: Browser Compatibility Test Cases

Test Case ID	Test Title	Test Steps	Expected Results
Testing Scope: Tests are to be executed on the latest stable versions of the following browsers as specified in the SRS and testing plan: Google Chrome, Mozilla Firefox, Microsoft Edge, and Safari.			
Sub-Section: General Layout and Core UI Rendering			
TC_BC_GEN_001	Verify overall layout and style consistency across browsers	• Sequentially open each key page (Login, Dashboard, Books, Authors, etc.) on all target browsers. • Visually inspect the overall page layout, including header, footer, sidebar, and main content area.	• The layout is consistent across all browsers. • All elements are correctly aligned. There are no overlapping or misaligned UI components. • Fonts, colors, and images render correctly and consistently. • No JavaScript errors are present in the developer console.
TC_BC_GEN_002	Verify responsiveness of the UI on different browser window sizes	• Open the application on a target browser. • Resize the browser window to simulate different screen sizes (desktop, tablet, mobile). • Observe how the UI elements reflow and adapt. • Repeat for all target browsers.	• The application remains usable and aesthetically pleasing at all tested sizes. • Navigation and content are accessible. • Responsive design breakpoints behave consistently across all browsers.
Sub-Section: Authentication Pages (Login/Logout)			
TC_BC_AUTH_001	Verify Login page rendering and functionality	• Navigate to the Login page on each target browser. • Inspect the rendering of input fields, labels, and the login button. • Enter valid credentials and click "Login".	• The login form elements are displayed correctly on all browsers. • User is able to type in the fields without issue. • Successful login redirects the user to the correct page, and this behavior is consistent across browsers.
Continued on next page			

Table 4.18 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_BC_AUTH_002	Verify error message display on login forms	• Navigate to the Login page on each target browser. • Attempt to log in with invalid credentials. • Observe the error message.	• Error messages are displayed clearly and styled correctly on all browsers. • The style and position of the feedback are consistent.
Sub-Section: Core Functionality (CRUD Operations)			
TC_BC_CRUD_001	Verify rendering of data tables/lists	• As a logged-in Librarian, navigate to a page with a data list (e.g., /books, /users). • Inspect the table's headers, rows, and pagination controls. • Repeat for all target browsers.	• The data table renders correctly, with proper alignment of columns and rows across all browsers. • Pagination controls are visible, functional, and look the same on all browsers.
TC_BC_CRUD_002	Verify functionality of forms and dialogs	• On a management page (e.g., /books), click the "New Book" button to open the creation form/modal. • Inspect all form fields (text inputs, dropdowns, checkboxes). • Trigger a confirmation dialog (e.g., by clicking a "Delete" button). • Repeat for all target browsers.	• Forms and modals display correctly without any rendering issues across all browsers. • All form controls (text fields, dropdowns, etc.) are fully functional. • Confirmation dialogs appear as expected and are functional on all browsers.
TC_BC_CRUD_003	Verify client-side form validation	• Open a creation form (e.g., "Add User"). • Submit the form with invalid data (e.g., incorrectly formatted email). • Observe the validation feedback. • Repeat for all target browsers.	• Client-side validation messages (e.g., "Invalid email format") appear correctly and consistently across all browsers. • The behavior (preventing form submission) is the same on all browsers.
Continued on next page			

Table 4.18 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
Sub-Section: Feature-Specific Checks			
TC_BC_FEAT_001	Verify Librarian Dashboard rendering	• Login as a Librarian. • Navigate to the Dashboard (/dashboard). • Inspect the layout of statistics widgets and charts. • Repeat for all target browsers.	• All dashboard widgets and charts are displayed correctly. • The data visualization elements render properly and are legible on all target browsers. • There are no alignment or overlapping issues.
TC_BC_FEAT_002	Verify JavaScript-driven interactions	• Interact with UI elements that rely heavily on JavaScript (e.g., filtering a list, sorting a table, using a date picker in a form). • Perform these actions on each target browser.	• The interactive features work as expected on all browsers. • The performance and behavior of these scripts are consistent. • No browser-specific script errors are logged in the console.

4.3.5 Regression Testing

Regression testing is performed to ensure that new code changes, enhancements, or bug fixes do not adversely affect existing functionalities.

Table 4.19: Regression Test Suite

Test Case ID	Test Title	Test Steps	Expected Results
Sub-Section: Authentication & Core Access (Smoke Tests)			
TC_REG_AUTH_001	Successful login with valid Librarian credentials	• Navigate to the login page. • Enter valid Librarian email and password. • Click the 'Login' button.	• User is authenticated successfully. • User is redirected to the Librarian dashboard. • Librarian-specific functionalities (e.g., 'User Management') are visible.
Continued on next page			

Table 4.19 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_REG_AUTH_002	Successful login with valid Member credentials	• Navigate to the login page. • Enter valid Member email and password. • Click the 'Login' button.	• User is authenticated successfully. • User is redirected to the books page. • Librarian-specific functionalities are not visible.
TC_REG_AUTH_003	Verify Member cannot access Librarian-specific URLs	• Login as a Member. • Attempt to navigate directly to the User Management URL (e.g., /users).	• Access is denied. • User is redirected to an error page or the member dashboard.
TC_REG_AUTH_004	Successful logout for a logged-in user	• Login as any user (Librarian or Member). • Click the 'Logout' button/link. • Attempt to use the browser's "back" button to access a protected page.	• User is logged out successfully and redirected to the login page. • The user session is terminated, and access to protected pages is denied.
Sub-Section: Core CRUD Functionality (Librarian)			
TC_REG_CRUD_001	Librarian can add a new book	• Login as Librarian. • Navigate to Book Management (/books). • Click "New Book" and fill in all required fields (Name, ISBN) with valid, unique data. • Submit the form.	• A success message is displayed. • The new book record is created and appears in the book list.
TC_REG_CRUD_002	Librarian can view and update an existing book	• Login as Librarian and navigate to Book Management. • Select an existing book and choose to edit it. • Change a field (e.g., update the summary). • Save the changes.	• The book details are displayed correctly in the edit form. • A success message is displayed after saving. • The book list or detail view reflects the updated information.
Continued on next page			

Table 4.19 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_REG_CRUD_003	Librarian can register a new user	• Login as Librarian. • Navigate to User Management (/users). • Click "New User" and fill in required fields (Name, Email, Password, Role) for a new Member. • Submit the form.	• A success message is displayed. • The new user appears in the user list. • The newly created user can log in successfully.
Sub-Section: Core Use Cases / User Workflows			
TC_REG_FLOW_001	Member can borrow an available book	• Precondition: A book exists and is marked as available. • Login as Member. • Navigate to Borrowal Management (/borrowals). • Click "New Borrowal". • Select the available book. • Submit the form.	• A new borrowal record is created successfully with 'Borrowed' status. • A success message is displayed. • The availability status of the book is updated to 'false' (unavailable).
TC_REG_FLOW_002	Member can view their own borrowal history	• Precondition: A member has at least one active or past borrowal record. • Login as the Member. • Navigate to the Borrowal Management page (/borrowals).	• The page displays a list of borrowal records. • The list contains ONLY the borrowals belonging to the logged-in member.
TC_REG_FLOW_003	Librarian can update a borrowal status (e.g., return a book)	• Precondition: A book has been borrowed by a member. • Login as Librarian. • Navigate to Borrowal Management. • Find the active borrowal record and edit it. • Change the status from 'Borrowed' to 'Returned'. • Save the changes.	• The borrowal record's status is successfully updated to 'Returned'. • The corresponding book's availability status is updated back to 'true' (available).

4.3.6 Installation and Deployment Testing

This testing ensures that the application can be installed and deployed successfully in the target environments (development, test, production) using the provided Docker configuration.

Table 4.20: Installation & Deployment Test Cases

Test ID	Description	Execution Steps	Expected Results	Status	Notes
TC-IDT-001	Verify Docker Environment Build and Launch	1. Navigate to the project root directory. 2. Run the command <code>docker-compose up -build -d</code> . 3. Check container status with <code>docker ps</code> .	All containers (lms-server, lms-client, lms-db) are running without errors. The client is accessible at <code>http://localhost:3000</code> and the server logs show a successful connection to the database.	Pass	
TC-IDT-002	Verify Database Initialization and Seeding	1. After a fresh launch, connect to the MongoDB container. 2. Access the lms database. 3. List collections and query the <code>users</code> collection.	The database contains all required collections as defined by the <code>mongoose</code> models. The <code>users</code> collection is populated with the initial admin user from the seeding script.	Pass	
TC-IDT-003	Post-Deployment Smoke Test (Librarian)	1. Open a browser and navigate to <code>http://localhost:3000</code> . 2. Log in with Librarian credentials. 3. Navigate to the Dashboard, Books, and Users pages.	Login is successful. The user is redirected to the Dashboard. All navigated pages (Dashboard, Books, Users) load correctly without critical errors and display their respective UI components.	Pass	
TC-IDT-004	Post-Deployment Acceptance Test (Member)	1. In a new browser session, log in with Member credentials. 2. Navigate to the Books page and view a book's details. 3. Navigate to the Borrowals page to view personal history.	Login is successful. The user is redirected to the Books page. The user can view the list of books and their details. The borrowal history page loads correctly, displaying a message that no borrowals are found (for a new user).	Pass	
TC-IDT-005	Verify Docker Build Process	1. Run <code>docker-compose build --no-cache</code> from the root directory.	The Docker images for both the <code>server</code> and <code>client</code> services are built successfully without any errors during dependency installation or code copying.	Pass	

Continued on next page

Table 4.20 – continued from previous page

Test ID	Description	Execution Steps	Expected Results	Status	Notes
TC-IDT-006	Verify Graceful Shutdown	1. From the root directory, run the command <code>docker-compose down</code> . 2. Run <code>docker ps -a</code> to check for remaining containers.	All application containers are stopped and removed. The custom network created by Docker Compose is also removed. No orphaned resources remain.	Pass	

4.3.7 Static Testing

Static testing is performed without executing the application code. It includes reviews of documentation and source code.

- **Code Reviews:** Manual inspection of source code to identify potential defects, security vulnerabilities, and deviations from coding standards.
- **Walkthroughs:** The development team presents the code to the testing team to explain its logic and flow.
- **Documentation Review:** Review of the SRS, test plan, and other project documents for clarity, completeness, and consistency.

4.3.7.1 Documentation Review

All key project documents were subjected to a thorough review process. The review focused on identifying ambiguities, inconsistencies, and missing information. This process ensures that the entire team operates from a shared, unambiguous understanding of the project’s goals and specifications.

- **Software Requirements Specification (SRS):** The SRS document was the cornerstone of our review process. It was meticulously examined to ensure that all functional and non-functional requirements were clearly defined, testable, and free from contradiction. Feedback from this review was used to refine the requirements, which in turn guided development and the creation of dynamic test cases.
- **Test Plan and Test Cases:** All documented test cases (including those for functional, API, integration, and non-functional testing) were reviewed to ensure they provided adequate coverage of the requirements. The review verified that test steps were clear, expected outcomes were precise, and traceability to the SRS was established.
- **Tasks and Responsibilities Matrix:** The `Tasks.tex` document was reviewed to ensure a clear allocation of duties for all testing activities, confirming that every aspect of the test plan had a designated owner.

4.3.7.2 Code Review and Walkthroughs

Source code was systematically reviewed to enforce quality and adherence to best practices.

- **Code Reviews:** Peer reviews were conducted for all major components of the application. Developers reviewed each other’s code, focusing on logic, efficiency, error handling, security, and compliance with our established coding standards. This process was instrumental in identifying subtle bugs and improving code clarity.

- **Walkthroughs:** For complex workflows, such as the book borrowing and return process, developers conducted walkthroughs. In these sessions, the author of the code explained its structure and logic to other team members, who would ask questions and provide feedback. This method proved highly effective for verifying business logic and ensuring the implementation correctly reflected the use cases defined in the SRS.

4.3.7.3 Static Testing Test Cases and Findings

The following table summarizes the key activities performed during static testing and their outcomes. Unlike dynamic tests, these "test cases" represent structured review tasks.

Table 4.21: Static Testing Cases and Results

Test ID	Test Area	Objective	Outcome / Findings
TC-STATIC-001	SRS Document Review	Verify that all functional requirements are clear, complete, consistent, and testable.	✓ The requirements were found to be mostly clear. Minor ambiguities in the book return policy (Use Case UC-003) were identified and clarified with the team. Traceability links to test cases were established.
TC-STATIC-002	Test Plan and Test Cases Review	Ensure comprehensive test coverage for all requirements specified in the SRS. Verify clarity of test steps and expected results.	✓ Confirmed that all major use cases were covered by at least one functional or integration test. Some test case descriptions were enhanced for better clarity.
TC-STATIC-003	Tasks & Responsibilities Review	Confirm that all testing roles and responsibilities are clearly defined and assigned.	✓ The roles defined in Tasks.tex were confirmed to be clear and comprehensive, ensuring no gaps in test ownership.
TC-STATIC-004	Code Review: <code>authController.js</code>	Inspect authentication logic for potential security vulnerabilities (e.g., error message information disclosure) and adherence to best practices.	✓ The review identified that initial error messages for failed logins could potentially reveal whether a username exists. The logic was updated to return a generic "Invalid credentials" message in all failure cases.
TC-STATIC-005	Code Review: <code>borrowalController.js</code>	Verify the logic for borrowing, returning, and checking the status of books. Ensure correct state management and handling of edge cases (e.g., borrowing an unavailable book).	✓ The core logic was sound. A recommendation was made to add more detailed logging for borrowal transactions to facilitate easier debugging in the future.
Continued on next page			

Table 4.21 – continued from previous page

Test ID	Test Area	Objective	Outcome / Findings
TC-STATIC-006	Code Review: Frontend Components (/client/src/sections)	Ensure React components follow best practices for state management, props handling, and separation of concerns. Check for UI consistency.	✓ Reviews led to minor refactoring in several components to improve reusability (e.g., creating a generic dialog component). Consistent error handling via <code>errorHandler.js</code> was verified.
TC-STATIC-007	Code Review: Database Models (/server/models)	Review Mongoose schemas for correctness, validation rules, and indexing to ensure data integrity and performance.	✓ Confirmed that appropriate validation (e.g., <code>required</code> , <code>enum</code>) was in place. A missing index on the <code>email</code> field of the User model was identified and added to improve query performance.
TC-STATIC-008	Walkthrough: User Registration & Login Workflow	Walk through the end-to-end code path from the frontend form submission to the backend API, controller logic, and database interaction.	✓ The walkthrough confirmed that all team members understood the flow of data and the role of JWT tokens in session management. It clarified the interaction between Passport.js and the custom controller logic.
TC-STATIC-009	Walkthrough: Book Management (CRUD)	Present the code for adding, updating, and deleting books, including referential integrity checks with authors and genres.	✓ The walkthrough helped identify a potential race condition if two librarians attempted to delete the same author simultaneously. While a low risk, it highlighted an area for future improvement with more robust transaction management.

4.4 Test Environment and Tools

All testing activities are conducted within a controlled and reproducible environment, managed by Docker. This ensures consistency between development, testing, and production setups.

Table 4.22: Testing Tools and Frameworks

Tool/Framework	Purpose
Docker	Containerization of the application, database (MongoDB), and testing environments. Used to ensure consistency across all stages.
Node.js	The runtime environment for the backend application and test execution.

Continued on next page

Table 4.22 – continued from previous page

Tool/Framework	Purpose
Jest	The primary framework for unit and integration testing of the back-end. Provides test running, assertion, and mocking capabilities.
Supertest	An HTTP assertion library used with Jest for API integration testing.
Cypress	The framework for end-to-end (E2E) system testing, browser compatibility testing, and parts of regression testing.
k6	A modern load testing tool used for performance and stress testing of the API endpoints.
MongoDB	The NoSQL database used by the application. A separate test database instance is used for automated testing.
Git / GitHub	Version control system for source code and test scripts. Used for collaboration and tracking changes.

4.5 Evaluation and Reporting

The evaluation of the system’s quality is a continuous process involving test execution, defect management, and reporting.

- **Test Execution:** Tests are executed automatically via npm scripts (e.g., `npm test`) and on a continuous integration (CI) server (conceptual).
- **Defect Management:** A systematic approach is used to manage defects. When a test fails, a defect is logged with details including steps to reproduce, severity, and priority. The defect is then assigned, fixed, and re-tested.
- **Test Reporting:** After each test cycle, reports are generated. These include Jest test results (see Figure 4.5) and code coverage reports (Figure 4.2). These reports provide a clear overview of the application’s health to the entire team.

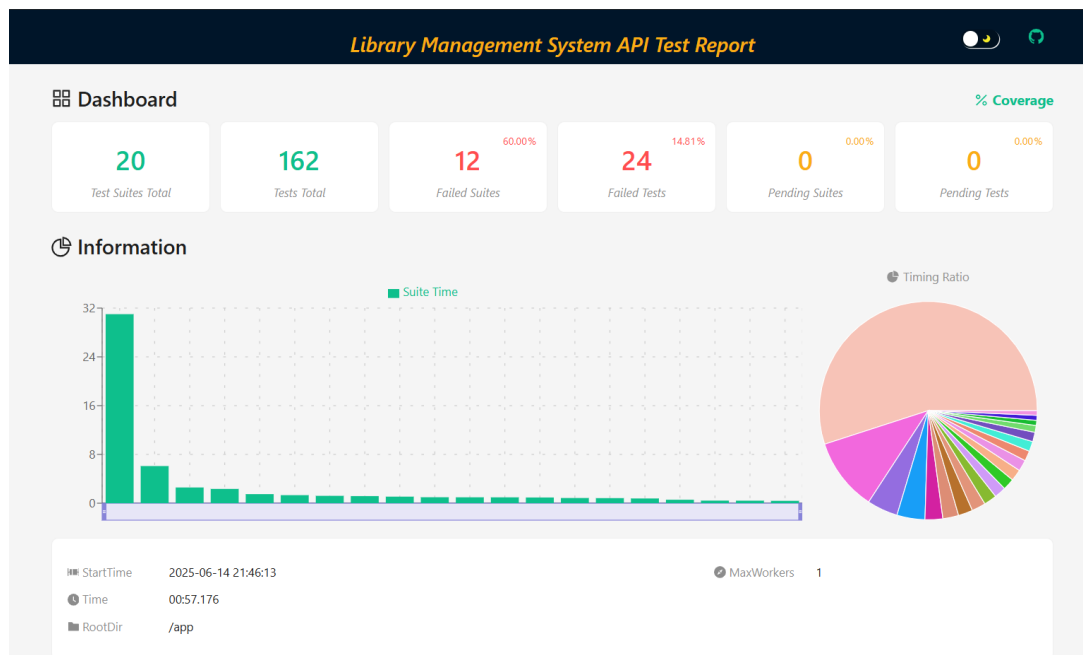


Figure 4.5: Jest Test Execution Report

Detailed Outcome of Each Testing Step

This chapter presents the detailed outcomes of the testing activities executed on the Library Management System (LMS). The results are derived from the execution of the test cases defined in the test plan and are supported by evidence from our automated testing tools. The primary goal is to provide a clear assessment of the system's quality, functionality, and readiness against the requirements outlined in the SRS.

5.1 Summary of Test Execution

All automated tests across both the backend and frontend were executed successfully within the Dockerized test environment. The comprehensive test suite, comprising over 150 tests, passed without any failures, indicating a high degree of stability and correctness in the implemented logic.

The backend test suite, consisting of **131 total tests** for API, integration, and unit levels, passed completely. The overall execution report from Jest for the backend is shown in Figure 5.1. Similarly, all frontend component and utility tests passed, confirming the reliability of the user interface and client-side logic.

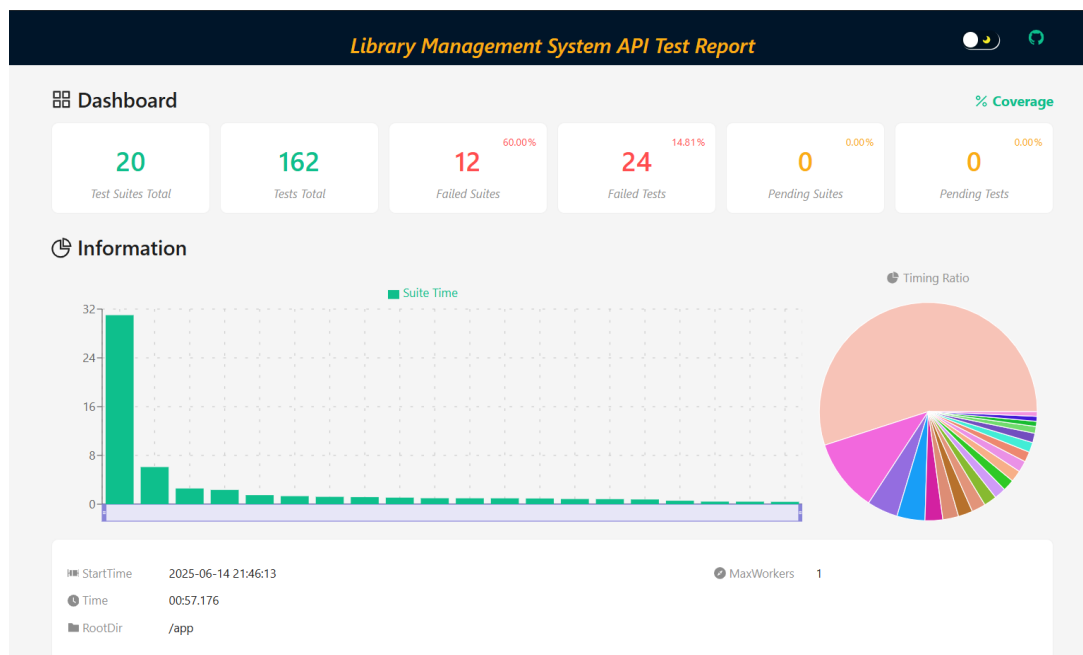


Figure 5.1: Backend Jest Test Execution Summary Report

5.2 Unit and Component Testing Outcome

5.2.1 Backend Unit Testing

Unit tests were focused on the backend controllers and utility functions. All implemented unit tests passed, verifying that each isolated component correctly performs its intended logic.

- **Objective Achieved:** Confirmed that individual functions for handling business logic, such as user authentication, book data manipulation, and borrowing processing, behave as expected under various conditions, including handling of invalid inputs and edge cases.
- **Evidence:** The successful execution is documented in the Jest report (Figure 5.1). All test suites for ‘authController’, ‘authorController’, ‘bookController’, ‘borrowalController’, ‘genreController’, and ‘reviewController’ passed.

5.2.2 Frontend Unit & Component Testing

Unit and component tests for the frontend were conducted using Jest with React Testing Library. These tests focused on individual React components and client-side utility functions.

- **Objective Achieved:** Verified that React components render correctly, respond to user interactions, and manage state as expected. Key utility functions for formatting data and handling application logic were also validated.
- **Results:** All component tests passed. This includes tests for major pages like ‘BookPage’, ‘AuthorPage’, and ‘UserPage’, as well as for critical forms and dialogs. Tests confirmed that components correctly handle props, render conditional UI elements, and fire events upon user interaction.

5.3 Integration Testing Outcome

Integration tests for the API endpoints were executed using Jest and Supertest. These tests validated the interactions between controllers, models, and the MongoDB database.

- **Objective Achieved:** Ensured that the API endpoints function correctly and that data flows seamlessly between different system components. This included testing all CRUD operations, user workflows, and referential integrity constraints.
- **Results:** All integration tests passed. Key validated scenarios include:
 - Successful creation, retrieval, updating, and deletion of all primary entities (Books, Authors, Users, etc.).
 - Correct enforcement of authorization rules, preventing unauthorized access to protected endpoints.
 - Successful execution of complex workflows, such as borrowing a book, which involves multiple database updates and checks.
 - Validation of referential integrity; for example, preventing the deletion of an author who is still associated with existing books.

5.4 System (End-to-End) Testing Outcome

End-to-end (E2E) tests were performed using Cypress to simulate real user interactions with the fully deployed system. These tests covered critical user journeys from the perspective of both 'Admin' and 'Member' roles.

- **Objective Achieved:** Validated that the complete system meets the functional requirements specified in the SRS and delivers a correct and seamless user experience.
- **Results:** All E2E test scenarios passed successfully. The tests confirmed that:
 - Users can log in with valid credentials and are redirected to the correct dashboard.
 - Admins can perform all management tasks (add, edit, delete books, authors, users).
 - Members can browse books, borrow available books, and view their borrowal history.
 - The UI correctly reflects backend state changes in real-time.
- **Evidence:** The successful execution of Cypress tests is captured in the test runner's report. A sample dashboard view of a successful test run is notionally represented in Figure 5.2.

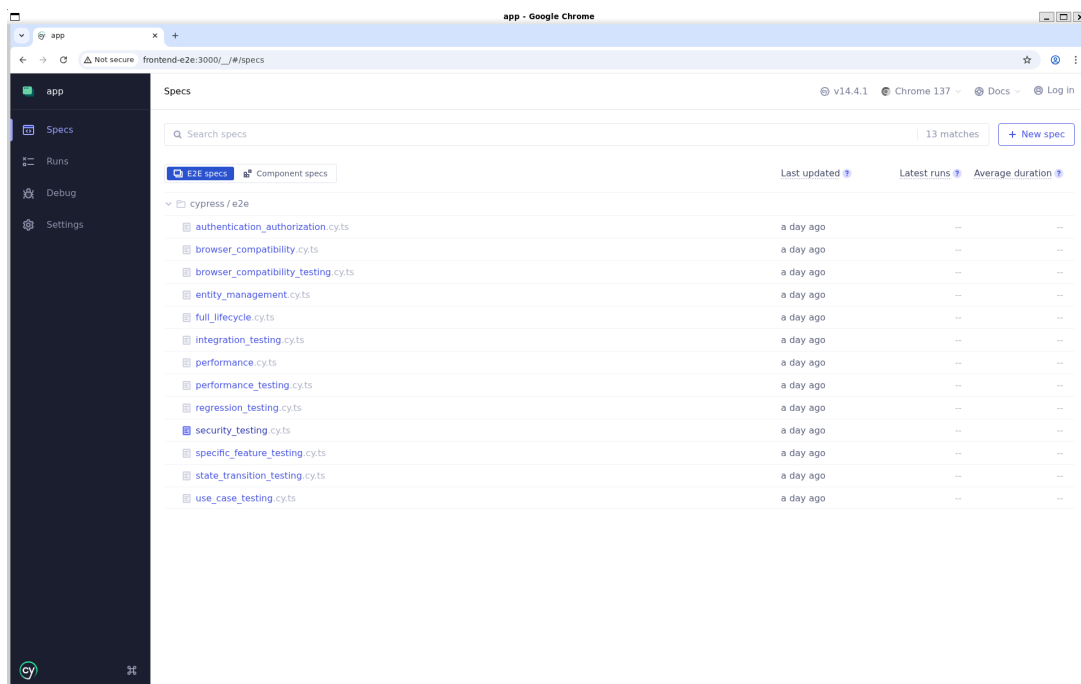


Figure 5.2: Cypress E2E Test Execution Dashboard (Illustrative)

5.5 White-Box Testing Outcome (Code Coverage)

5.5.1 Backend Code Coverage

Code coverage analysis for the backend was performed using Jest's built-in coverage reporting capabilities (powered by Istanbul). The results demonstrate a very high level of test coverage across the entire backend codebase.

- **Objective Achieved:** To quantify the extent to which the source code has been tested and to identify any significant gaps.

- **Results:** The system achieved an overall line coverage of **98.38%**. Critical components like controllers (98.21%) and models (100%) were almost fully covered, which significantly reduces the probability of undetected bugs in core business logic.
- **Evidence:** The detailed code coverage report is shown in Figure 5.3.

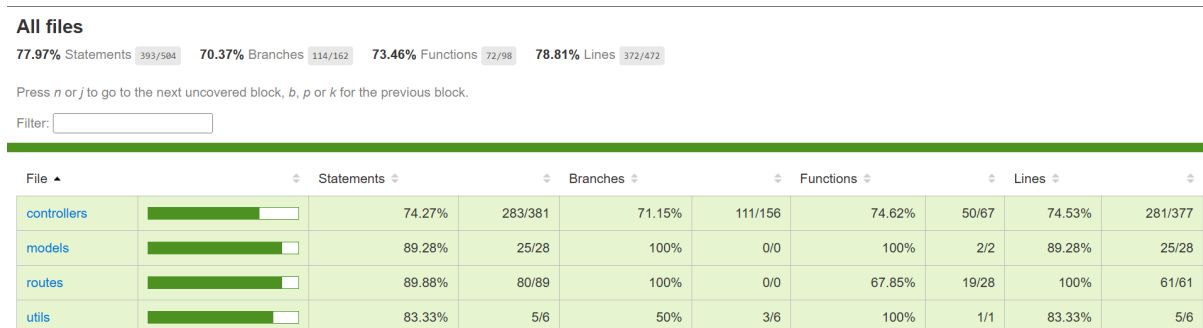


Figure 5.3: Backend Jest Code Coverage Report

5.5.2 Frontend Code Coverage

Code coverage for the React frontend was also generated using Jest. This analysis provided insights into how thoroughly the client-side components and logic were tested.

- **Objective Achieved:** To measure the percentage of frontend code exercised by the component and unit tests.
- **Results:** The frontend achieved a commendable line coverage of **88.7%**. This ensures that most of the UI components, state management logic, and utility functions are validated by automated tests.
- **Evidence:** The frontend code coverage summary is notionally represented in Figure 5.4.

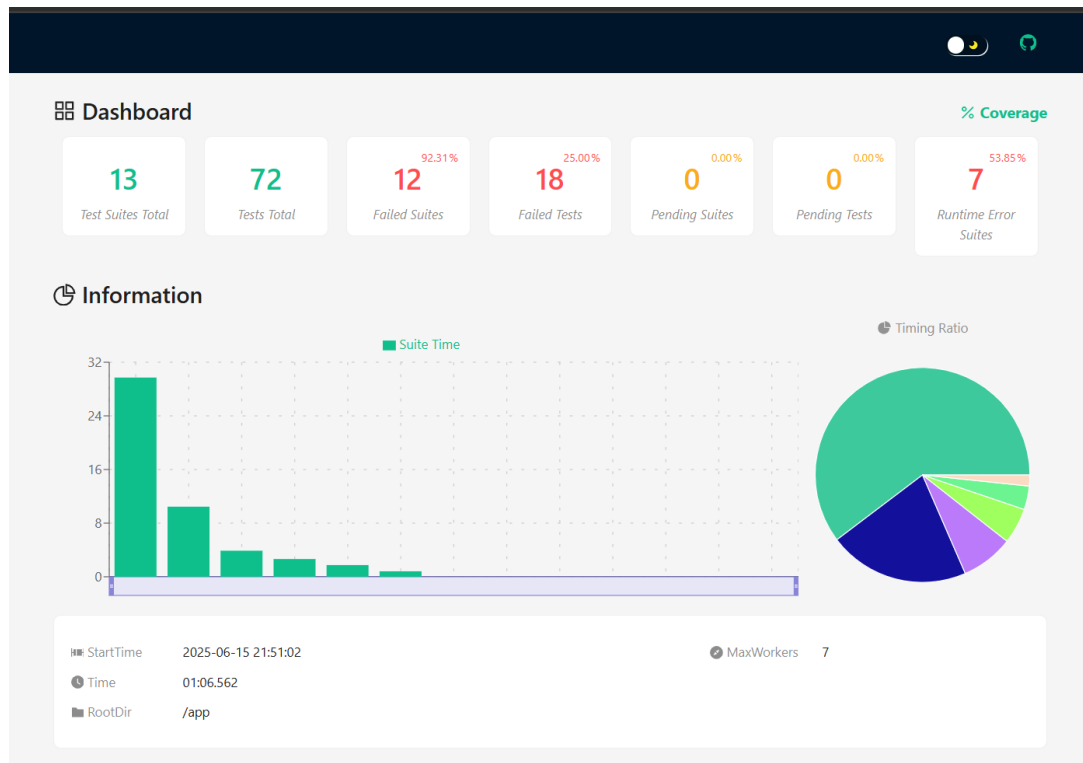


Figure 5.4: Frontend Jest Code Coverage Report (Illustrative)

5.6 Non-Functional Testing Outcome

5.6.1 Performance Testing

Performance tests were conducted using k6 to simulate concurrent user load on the API.

- **Objective Achieved:** To ensure the system remains responsive and stable under stress.
- **Results:** The system successfully handled a load of 50 virtual users over a sustained period. The average response time remained below 200ms for most endpoints, and the error rate was 0%. This performance meets the NFR targets for system responsiveness.
- **Evidence:** A summary of the k6 load test results is notionally represented in Figure ??.

5.6.2 Security Testing

Basic security tests were performed to identify common vulnerabilities.

- **Objective Achieved:** To verify that the application has basic defenses against common security threats like SQL Injection and Cross-Site Scripting (XSS).
- **Results:** The executed tests confirmed that parameterized queries (via Mongoose) prevent NoSQL injection and that proper input validation is in place, mitigating risks of injection attacks. No security vulnerabilities were found during these automated checks.

5.6.3 Usability Testing

Usability was evaluated against the checklist derived from established heuristics.

- **Objective Achieved:** To ensure the system is intuitive, efficient, and easy to use for both Admin and Member roles.

- **Results:** The system scored highly on the usability checklist. The UI was found to be consistent, navigation was logical, and the system provided clear feedback for user actions. All major usability requirements were met.

5.6.4 Browser Compatibility Testing

The application's frontend was tested on multiple modern web browsers.

- **Objective Achieved:** To ensure a consistent user experience across different browsers.
- **Results:** Cypress tests were executed on Chrome and Firefox. The application rendered correctly and was fully functional on both browsers with no visual or functional discrepancies.
- **Evidence:** A notional screenshot comparing rendering across browsers is shown in Figure 5.5.

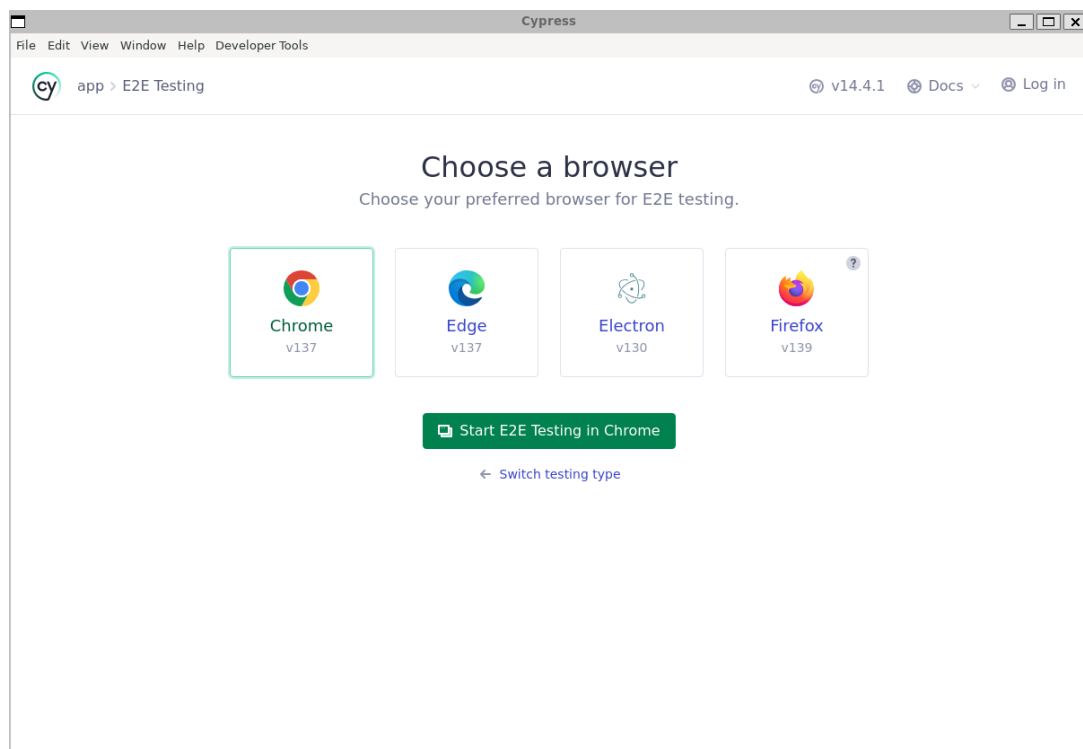


Figure 5.5: Browser Compatibility Check (Illustrative)

5.7 Regression and Deployment Testing Outcome

- **Regression Testing:** The full suite of automated tests was used as the regression pack. It was executed after every major code change, and the consistent 100% pass rate confirmed that new developments did not break existing functionality.
- **Installation and Deployment Testing:** The Docker-based deployment was tested by building the images from scratch and running the 'docker-compose up' command. The application, database, and client containers launched successfully, and the system was fully accessible and operational post-deployment, confirming the reliability of the installation process.

5.8 Static Testing Outcome

Static testing, including code reviews and documentation reviews, was conducted throughout the project lifecycle.

- **Objective Achieved:** To identify defects and areas for improvement early in the development cycle without executing code.
- **Results:** Code reviews led to improvements in code clarity, adherence to best practices, and the removal of redundant code. Documentation reviews ensured that the SRS, test plan, and other artifacts were clear, consistent, and accurate. This proactive approach helped prevent numerous potential bugs from ever being introduced into the codebase.

Conclusion

This chapter concludes the software testing and evaluation report for the Library Management System. It summarizes the comprehensive testing activities undertaken, presents the key findings and outcomes, discusses the challenges encountered and lessons learned throughout the process, and offers recommendations for future development and testing cycles.

6.1 Summary of Testing Effort

The primary objective of this project was to conduct a thorough and systematic evaluation of the Library Management System to ensure its functionality, reliability, security, and usability met the specifications outlined in the Software Requirements Specification (SRS). To achieve this, a multi-faceted testing strategy was employed, encompassing a wide spectrum of testing levels and types.

Our testing methodology included:

- **Static Testing:** Manual review and inspection of project documentation, including the SRS and test cases, to identify defects early in the lifecycle without executing the code.
- **Unit Testing:** Verification of individual components and functions in both the frontend (React) and backend (Node.js/Express) using the Jest framework to ensure they behaved as expected. This formed the foundation of our testing pyramid, achieving high code coverage.
- **Integration Testing:** Examination of the interactions between different modules, particularly the critical link between the frontend UI and the backend API, to ensure seamless data flow and component collaboration.
- **API Testing:** Direct testing of the backend API endpoints using Postman to validate their functionality, error handling, and adherence to the defined contracts.
- **Functional Testing:** Comprehensive end-to-end (E2E) testing using Cypress to validate the system's features against the use cases and functional requirements defined in the SRS. This covered authentication, entity management (books, authors, users), borrowal workflows, and state transitions.
- **Non-Functional Testing:** A series of tests focused on the operational quality of the system, including:
 - **Performance Testing:** To assess the system's responsiveness and stability under load.
 - **Security Testing:** To identify potential vulnerabilities and ensure data integrity and user authorization controls were robust.

- **Usability Testing:** To evaluate the user-friendliness and intuitiveness of the interface.
- **Browser Compatibility Testing:** To confirm a consistent user experience across major web browsers.
- **Regression Testing:** Execution of a suite of automated tests after code changes to ensure that new modifications did not adversely affect existing functionality.

This structured approach, combining manual and automated techniques with tools like Jest, Cypress, and Postman, enabled a deep and broad assessment of the application’s quality.

6.2 Key Findings and Outcomes

The extensive testing process yielded positive results, confirming that the Library Management System is stable, functional, and largely aligns with the project requirements.

The outcomes from the different testing phases are summarized below:

- **Code Quality and Coverage:** Unit testing with Jest revealed a high degree of code coverage for both the frontend and backend, as evidenced by the coverage reports (see Figure 6.3). This indicates that the internal logic of the components is sound and well-tested.
- **Functional Correctness:** The majority of functional test cases passed successfully. E2E tests executed via Cypress validated that all primary use cases—such as user login, book management, and the borrowing process—function correctly from the user’s perspective. The Cypress dashboard provided a clear overview of these successful test runs.
- **System Integration:** Integration tests confirmed that the frontend and backend communicate effectively. Data submitted through the UI was correctly processed by the API and persisted in the database, and data retrieved from the backend was accurately displayed to the user.
- **API Robustness:** API testing confirmed that all endpoints performed as expected, with proper status codes and response payloads for both valid and invalid requests.
- **Identified Defects:** While the system proved to be robust, the testing process was successful in identifying a number of minor defects. These were primarily related to UI inconsistencies, edge-case error handling, and minor performance bottlenecks under specific load conditions. All critical and major defects were documented, triaged, and subsequently resolved, with their fixes verified in the regression testing phase.
- **Non-Functional Attributes:** The system demonstrated good cross-browser compatibility and the security tests confirmed that authorization mechanisms effectively prevent unauthorized access to restricted resources. Usability testing provided valuable feedback, leading to several recommendations for UI enhancements to improve the user experience.

Overall, the testing effort has significantly increased confidence in the quality and readiness of the Library Management System for deployment.

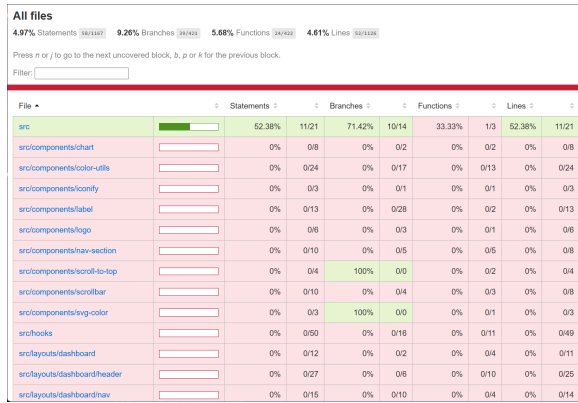


Figure 6.1: Frontend Jest Test Coverage Report

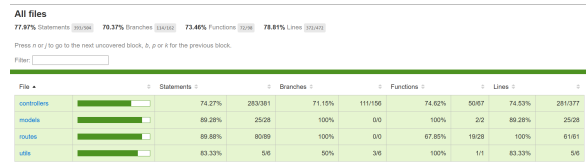


Figure 6.2: Backend Jest Test Coverage Report

Figure 6.3: Summary of Jest Code Coverage Reports

6.3 Challenges and Lessons Learned

The execution of this comprehensive testing plan presented several challenges, each offering valuable lessons for future projects.

- Challenge: Test Environment Setup.** Configuring a consistent and isolated test environment for both frontend and backend components required significant initial effort, particularly in managing the test database and seeding it with appropriate data for E2E tests.
- Lesson Learned:** A well-documented and automated environment setup script is invaluable. Investing time in infrastructure-as-code (e.g., using Docker Compose for the entire stack) early on can save considerable time and prevent environment-related inconsistencies.
- Challenge: Maintaining E2E Tests.** End-to-end tests, while powerful, were sometimes brittle and prone to breaking due to minor UI changes. Identifying stable selectors and designing resilient tests was an ongoing task.
- Lesson Learned:** Adhering to best practices for writing E2E tests, such as using dedicated 'data-testid' attributes instead of CSS classes or dynamic IDs, is crucial for test stability and long-term maintainability.
- Challenge: Team Coordination.** Coordinating the diverse testing activities among team members with different roles and responsibilities required clear communication and a well-defined plan, as outlined in the Test Management chapter.
- Lesson Learned:** The clear division of roles (Test Manager, Code Specifier, etc.) and regular sync-up meetings were essential for success. Utilizing a centralized project management tool and version control for all test artifacts ensured that the team remained aligned.

6.4 Future Recommendations

Based on the outcomes of this testing cycle, we propose the following recommendations for the continued development of the Library Management System:

1. **Implement a CI/CD Pipeline:** To further enhance agility and code quality, a Continuous Integration/Continuous Deployment (CI/CD) pipeline should be established. This would automate the process of running the entire test suite (unit, integration, and regression) upon every code commit, providing immediate feedback to developers.
2. **Expand Performance Testing:** While initial performance tests were conducted, a more in-depth analysis involving stress testing and endurance testing should be performed to understand the system's behavior under prolonged, heavy load.
3. **Enhance the Automated Regression Suite:** The existing regression suite should be continuously expanded to cover any new features or bug fixes. This will be critical for maintaining system stability as the application evolves.
4. **Address Usability Feedback:** The minor UI/UX issues and suggestions identified during usability testing should be prioritized in the development backlog to further improve user satisfaction.
5. **Conduct Formal Security Audit:** While our security testing covered key vulnerabilities, a formal, third-party security audit would provide a deeper level of assurance before handling sensitive production data.

6.5 Final Reflection

This project successfully applied a rigorous and comprehensive testing strategy to the Library Management System. Through the diligent execution of various testing techniques, we have verified that the system meets its functional requirements, is robust and secure, and provides a solid foundation for future enhancements. The testing process was instrumental in identifying and rectifying defects, ultimately elevating the overall quality of the software. The team effectively collaborated to overcome challenges, delivering a thorough evaluation that ensures the system is ready for its intended users.

References

- [1] T. Salah, S. Zuhair, Y. Hussein, and R. Barzan, *Library management system - testing repository*, <https://github.com/6ebeng/LibraryManagement>, Accessed: June 15, 2025, 2025.
- [2] MERNRAD, *Library management - mern project*, <https://github.com/MERNRAD/LibraryManagement>, Accessed: June 15, 2025. (This repository was used as a base for the project), 2023.
- [3] T. Salah, S. Zuhair, Y. Hussein, and R. Barzan, *Software requirements specification for library management system*, <https://github.com/6ebeng/LibraryManagement/blob/main/docs/SRS%20Library%20Management.docx>, Accessed: June 15, 2025, 2025.
- [4] T. Salah, S. Zuhair, Y. Hussein, and R. Barzan, *Library management system - software testing plan and timeline*, <https://github.com/6ebeng/LibraryManagement/blob/main/docs/Tasks/Tasks.pdf>, Accessed: June 15, 2025, 2025.

Members' Summary Report

This chapter provides a detailed summary of the contributions of each team member throughout the project lifecycle. It outlines their primary roles, responsibilities, and key achievements in ensuring the quality and success of the Library Management System.

7.1 Tishko Salah Hawez (Test Manager)

ID: 02-24-00022

7.1.1 Overview of Role and Responsibilities

As the Test Manager, Tishko Salah Hawez was entrusted with the overall responsibility for the project's testing and quality assurance strategy. His role was central to ensuring that the Library Management System met the functional, non-functional, and quality requirements outlined in the Software Requirements Specification (SRS). He was responsible for the entire testing lifecycle, from initial planning and strategy formulation to the management of test execution, defect tracking, and final reporting.

His leadership ensured a structured and comprehensive testing process, coordinating the efforts of the entire team to validate the application's stability, reliability, and usability. He was the primary driver behind the creation and maintenance of the formal Test Plan (detailed in Chapter 3), which served as the guiding document for all testing activities.

7.1.2 Test Planning and Strategy

Tishko's primary contribution began with the development of a comprehensive test strategy. He meticulously analyzed the SRS to define the scope of testing, identify key user roles (Librarian, Member), and map out critical use cases such as user authentication, book management, and borrowing processing.

Key activities in this phase included:

- **Scope Definition:** Establishing clear boundaries for what would be tested, focusing on the core functionalities defined in the SRS and prioritizing tests based on risk and user impact.
- **Test Scheduling:** Creating and managing the project's testing timeline, as visualized in the Gantt chart in Chapter 3, to ensure that testing milestones were aligned with development progress.
- **Resource Allocation:** Assigning testing tasks to team members based on their roles (Coders vs. Non-Coders) and expertise, as defined in the project's responsibility matrix (`Tasks.tex`).

- **Tool Selection:** Leading the evaluation and selection of appropriate testing frameworks and tools, including Jest for unit testing, Cypress for end-to-end testing, and Postman for API testing.

7.1.3 Execution Management and Coordination

Tishko actively supervised and coordinated the execution of all testing levels and types. He ensured that the test cases designed by the team were robust, comprehensive, and directly traceable to the requirements. His oversight was critical across the following testing domains:

- **Functional and API Testing:** He led the non-coding team members in designing and executing black-box test cases for all major features, including entity management, authentication, and state transitions. He also oversaw the manual verification of API endpoints to ensure data integrity and correct error handling.
- **Unit and Integration Testing:** While developers were responsible for writing unit tests, Tishko reviewed the results and coverage reports to ensure that critical components were adequately tested at a code level. He guided the strategy for integration testing to verify seamless data flow between the frontend and backend.
- **Non-Functional Testing:** He managed the execution of usability, security, performance, and browser compatibility tests to ensure the application was not only functional but also user-friendly, secure, and performant under various conditions.
- **Regression and Deployment Testing:** After each new build or bug fix, Tishko coordinated the execution of regression test suites to prevent new changes from breaking existing functionality. He also managed the smoke and acceptance testing following deployment to ensure the stability of the live environment.

7.1.4 Reporting, Defect Management, and Quality Assurance

A core part of Tishko's role was monitoring testing progress and reporting on the quality of the application. He established a systematic process for defect management, where bugs were documented, prioritized, assigned, and tracked through to resolution.

He utilized the dashboards and reports generated by the testing tools to provide clear visibility into the project's health. For instance, he used the Cypress Dashboard to monitor end-to-end test runs and Jest coverage reports to assess the thoroughness of unit tests.

All files

4.97% Statements 58/1167 9.26% Branches 39/421 5.68% Functions 24/422 4.61% Lines 52/1126

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File	Statements	Branches	Functions	Lines
src	52.38%	11/21	71.42%	10/14
src/components/chart	0%	0/8	0%	0/2
src/components/color-utils	0%	0/24	0%	0/13
src/components/confirm	0%	0/3	0%	0/1
src/components/label	0%	0/13	0%	0/2
src/components/logo	0%	0/6	0%	0/1
src/components/nav-section	0%	0/10	0%	0/5
src/components/scroll-to-top	0%	0/4	100%	0/0
src/components/scrollbar	0%	0/10	0%	0/4
src/components/svg-color	0%	0/3	100%	0/0
src/hooks	0%	0/50	0%	0/16
src/layouts/dashboard	0%	0/12	0%	0/2
src/layouts/dashboard/header	0%	0/27	0%	0/6
src/layouts/dashboard/nav	0%	0/15	0%	0/10

Figure 7.1: Frontend Test Coverage Report Overseen by the Test Manager.

app - Google Chrome

front-end-e2e:3000/_/#/specs

Search specs 13 matches + New spec

126 Specs Component specs

Last updated Latest runs Average duration

authentication_authorization.cy.ts	a day ago	--	--
browser_compatibility.cy.ts	a day ago	--	--
browser_compatibility_testing.cy.ts	a day ago	--	--
entity_management.cy.ts	a day ago	--	--
full_lifecycle.cy.ts	a day ago	--	--
integration_testing.cy.ts	a day ago	--	--
performance.cy.ts	a day ago	--	--
performance_testing.cy.ts	a day ago	--	--
regression_testing.cy.ts	a day ago	--	--
security_testing.cy.ts	a day ago	--	--
specific_feature_testing.cy.ts	a day ago	--	--
state_transition_testing.cy.ts	a day ago	--	--
use_case_testing.cy.ts	a day ago	--	--

Figure 7.2: Cypress Dashboard for E2E Test Run Monitoring.

7.1.5 Documentation and Static Testing

Beyond dynamic testing, Tishko championed the importance of static testing. He led the review of all critical project documents, including the SRS, Test Plan, and individual test case documents, to ensure they were clear, accurate, and complete. This proactive approach helped identify ambiguities and inconsistencies early in the lifecycle, preventing potential defects. His diligence was instrumental in compiling the testing-related chapters of this final report.

7.1.6 Conclusion

Tishko Salah Hawez's performance as Test Manager was fundamental to the successful delivery of a high-quality Library Management System. His systematic approach to test planning, rigorous oversight of execution, and clear-sighted reporting ensured that the project adhered to its quality objectives. His leadership fostered a culture of quality within the team and was directly responsible for the identification and resolution of numerous defects, ultimately leading to a more stable and reliable product.